

Assignment 2 --- Fraud Detection

1. Import Library

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# split data into train & test
from sklearn.model_selection import train_test_split

# Disable Warnings
import warnings
warnings.filterwarnings('ignore')
```

1. Read DataSet

```
url =
"https://raw.githubusercontent.com/ammishra08/Machine-Learning/refs/heads/master/Datasets/fraud_oracle.csv"

df1 = pd.read_csv(url)

print("Dataset loaded successfully!")

Dataset loaded successfully!
```

1. Displaying First 10 Rows

```
df1.head(10)
```

	Month	WeekOfMonth	DayOfWeek	Make	AccidentArea	DayOfWeekClaimed
0	Dec	5	Wednesday	Honda	Urban	Tuesday
1	Jan	3	Wednesday	Honda	Urban	Monday
2	Oct	5	Friday	Honda	Urban	Thursday
3	Jun	2	Saturday	Toyota	Rural	Friday
4	Jan	5	Monday	Honda	Urban	Tuesday
5	Oct	4	Friday	Honda	Urban	Wednesday
6	Feb	1	Saturday	Honda	Urban	Monday
7	Nov	1	Friday	Honda	Urban	Tuesday
8	Dec	4	Saturday	Honda	Urban	Wednesday

9	Apr	3	Tuesday	Ford	Urban	Wednesday
MonthClaimed		WeekOfMonthClaimed		Sex	MaritalStatus	...
AgeOfVehicle \						
0	Jan	1	Female	Single	...	3
years						
1	Jan	4	Male	Single	...	6
years						
2	Nov	2	Male	Married	...	7
years						
3	Jul	1	Male	Married	...	more
than 7						
4	Feb	2	Female	Single	...	5
years						
5	Nov	1	Male	Single	...	5
years						
6	Feb	3	Male	Married	...	7
years						
7	Mar	4	Male	Single	...	
new						
8	Dec	5	Male	Single	...	6
years						
9	Apr	3	Male	Married	...	more
than 7						
AgeOfPolicyHolder		PoliceReportFiled		WitnessPresent	AgentType \	
0	26 to 30		No	No	External	
1	31 to 35		Yes	No	External	
2	41 to 50		No	No	External	
3	51 to 65		Yes	No	External	
4	31 to 35		No	No	External	
5	21 to 25		No	No	External	
6	36 to 40		No	No	External	
7	16 to 17		No	No	External	
8	31 to 35		No	Yes	External	
9	36 to 40		No	No	External	
NumberOfSuppliments		AddressChange_Claim		NumberOfCars	Year	
BasePolicy						
0	none	1 year		3 to 4	1994	
Liability						
1	none	no change		1 vehicle	1994	
Collision						
2	none	no change		1 vehicle	1994	
Collision						
3	more than 5	no change		1 vehicle	1994	
Liability						
4	none	no change		1 vehicle	1994	
Collision						

5	3 to 5	no change	1 vehicle	1994	
Collision					
6	1 to 2	no change	1 vehicle	1994	
Collision					
7	none	no change	1 vehicle	1994	
Collision					
8	3 to 5	no change	1 vehicle	1994	
Collision					
9	3 to 5	no change	1 vehicle	1994	All
Perils					

[10 rows x 33 columns]

```
print("Dataset shape:", df1.shape)
```

Dataset shape: (15420, 33)

```
df1.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 15420 entries, 0 to 15419
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Month                                15420 non-null  str
1   WeekOfMonth                          15420 non-null  int64
2   DayOfWeek                            15420 non-null  str
3   Make                                 15420 non-null  str
4   AccidentArea                        15420 non-null  str
5   DayOfWeekClaimed                    15420 non-null  str
6   MonthClaimed                        15420 non-null  str
7   WeekOfMonthClaimed                  15420 non-null  int64
8   Sex                                 15420 non-null  str
9   MaritalStatus                       15420 non-null  str
10  Age                                  15420 non-null  int64
11  Fault                               15420 non-null  str
12  PolicyType                          15420 non-null  str
13  VehicleCategory                     15420 non-null  str
14  VehiclePrice                        15420 non-null  str
15  FraudFound_P                        15420 non-null  int64
16  PolicyNumber                        15420 non-null  int64
17  RepNumber                           15420 non-null  int64
18  Deductible                          15420 non-null  int64
19  DriverRating                        15420 non-null  int64
20  Days_Policy_Accident                 15420 non-null  str
21  Days_Policy_Claim                    15420 non-null  str
22  PastNumberOfClaims                   15420 non-null  str
23  AgeOfVehicle                         15420 non-null  str
24  AgeOfPolicyHolder                   15420 non-null  str
25  PoliceReportFiled                   15420 non-null  str
```

26	WitnessPresent	15420	non-null	str
27	AgentType	15420	non-null	str
28	NumberOfSuppliments	15420	non-null	str
29	AddressChange_Claim	15420	non-null	str
30	NumberOfCars	15420	non-null	str
31	Year	15420	non-null	int64
32	BasePolicy	15420	non-null	str

dtypes: int64(9), str(24)
memory usage: 3.9 MB

1. Check And Handling Missing Values

```
df1.isnull().sum()
```

Month	0
WeekOfMonth	0
DayOfWeek	0
Make	0
AccidentArea	0
DayOfWeekClaimed	0
MonthClaimed	0
WeekOfMonthClaimed	0
Sex	0
MaritalStatus	0
Age	0
Fault	0
PolicyType	0
VehicleCategory	0
VehiclePrice	0
FraudFound_P	0
PolicyNumber	0
RepNumber	0
Deductible	0
DriverRating	0
Days_Policy_Accident	0
Days_Policy_Claim	0
PastNumberOfClaims	0
AgeOfVehicle	0
AgeOfPolicyHolder	0
PoliceReportFiled	0
WitnessPresent	0
AgentType	0
NumberOfSuppliments	0
AddressChange_Claim	0
NumberOfCars	0
Year	0
BasePolicy	0

dtype: int64

```
# show descriptive summary of sample data
```

```
df1.describe()
```

	WeekOfMonth	WeekOfMonthClaimed	Age	FraudFound_P	\
count	15420.000000	15420.000000	15420.000000	15420.000000	
mean	2.788586	2.693969	39.855707	0.059857	
std	1.287585	1.259115	13.492377	0.237230	
min	1.000000	1.000000	0.000000	0.000000	
25%	2.000000	2.000000	31.000000	0.000000	
50%	3.000000	3.000000	38.000000	0.000000	
75%	4.000000	4.000000	48.000000	0.000000	
max	5.000000	5.000000	80.000000	1.000000	

	PolicyNumber	RepNumber	Deductible	DriverRating
Year				
count	15420.000000	15420.000000	15420.000000	15420.000000
mean	7710.500000	8.483268	407.704280	2.487808
std	4451.514911	4.599948	43.950998	1.119453
min	1.000000	1.000000	300.000000	1.000000
25%	3855.750000	5.000000	400.000000	1.000000
50%	7710.500000	8.000000	400.000000	2.000000
75%	11565.250000	12.000000	400.000000	3.000000
max	15420.000000	16.000000	700.000000	4.000000

```
df1.describe(include = 'str')
```

	Month	DayOfWeek	Make	AccidentArea	DayOfWeekClaimed
MonthClaimed	\				
count	15420	15420	15420	15420	15420
unique	12	7	19	2	8
top	Jan	Monday	Pontiac	Urban	Monday
freq	1411	2616	3837	13822	3757

	Sex	MaritalStatus	Fault	PolicyType	...	\
count	15420	15420	15420	15420	...	
unique	2	4	2	9	...	
top	Male	Married	Policy Holder	Sedan - Collision	...	
freq	13000	10625	11230	5584	...	

	PastNumberOfClaims	AgeOfVehicle	AgeOfPolicyHolder
PoliceReportFiled \			
count	15420	15420	15420
unique	4	8	9
top	2 to 4	7 years	31 to 35
No			
freq	5485	5807	5593
14992			

	WitnessPresent	AgentType	NumberOfSupplements
AddressChange_Claim \			
count	15420	15420	15420
unique	2	2	4
top	No	External	none
change			no
freq	15333	15179	7047
14324			

	NumberOfCars	BasePolicy
count	15420	15420
unique	5	3
top	1 vehicle	Collision
freq	14316	5962

[4 rows x 24 columns]

```
df1['DayOfWeekClaimed'].unique()
```

```
<StringArray>
[ 'Tuesday', 'Monday', 'Thursday', 'Friday', 'Wednesday',
 'Saturday',
 'Sunday', '0']
Length: 8, dtype: str
```

```
df1['MonthClaimed'].unique()
```

```
<StringArray>
['Jan', 'Nov', 'Jul', 'Feb', 'Mar', 'Dec', 'Apr', 'Aug', 'May', 'Jun',
 'Sep',
 'Oct', '0']
Length: 13, dtype: str
```

```
df1['Age'].unique()
```

```
array([0])
```

```
# Replacing 0 in Age column with the mean of non-zero values
df1['Age'] = df1['Age'].replace(0, df1.loc[df1['Age'] != 0,
'Age'].mean())
```

```
df1.describe()
```

	WeekOfMonth	WeekOfMonthClaimed	Age	FraudFound_P	\
count	15420.000000	15420.000000	15420.000000	15420.000000	
mean	2.788586	2.693969	40.700331	0.059857	
std	1.287585	1.259115	12.181090	0.237230	
min	1.000000	1.000000	16.000000	0.000000	
25%	2.000000	2.000000	31.000000	0.000000	
50%	3.000000	3.000000	39.000000	0.000000	
75%	4.000000	4.000000	48.000000	0.000000	
max	5.000000	5.000000	80.000000	1.000000	

	PolicyNumber	RepNumber	Deductible	DriverRating
Year				
count	15420.000000	15420.000000	15420.000000	15420.000000
mean	7710.500000	8.483268	407.704280	2.487808
std	4451.514911	4.599948	43.950998	1.119453
min	1.000000	1.000000	300.000000	1.000000
25%	3855.750000	5.000000	400.000000	1.000000
50%	7710.500000	8.000000	400.000000	2.000000
75%	11565.250000	12.000000	400.000000	3.000000
max	15420.000000	16.000000	700.000000	4.000000

```
print("Number of duplicate rows:", df.duplicated().sum())
```

```
Number of duplicate rows: 0
```

1. Separate numerical and categorical variables

```
numeric_columns =
df1.select_dtypes(include=np.number).columns.tolist()
categorical_columns =
df1.select_dtypes(include='object').columns.tolist()

print("Numerical Columns:")
print(numeric_columns)

print("\nCategorical Columns:")
print(categorical_columns)
```

Numerical Columns:

```
['WeekOfMonth', 'WeekOfMonthClaimed', 'Age', 'FraudFound_P',  
'PolicyNumber', 'RepNumber', 'Deductible', 'DriverRating', 'Year']
```

Categorical Columns:

```
['Month', 'DayOfWeek', 'Make', 'AccidentArea', 'DayOfWeekClaimed',  
'MonthClaimed', 'Sex', 'MaritalStatus', 'Fault', 'PolicyType',  
'VehicleCategory', 'VehiclePrice', 'Days_Policy_Accident',  
'Days_Policy_Claim', 'PastNumberOfClaims', 'AgeOfVehicle',  
'AgeOfPolicyHolder', 'PoliceReportFiled', 'WitnessPresent',  
'AgentType', 'NumberOfSupplements', 'AddressChange_Claim',  
'NumberOfCars', 'BasePolicy']
```

1. Exploratory Data Analysis

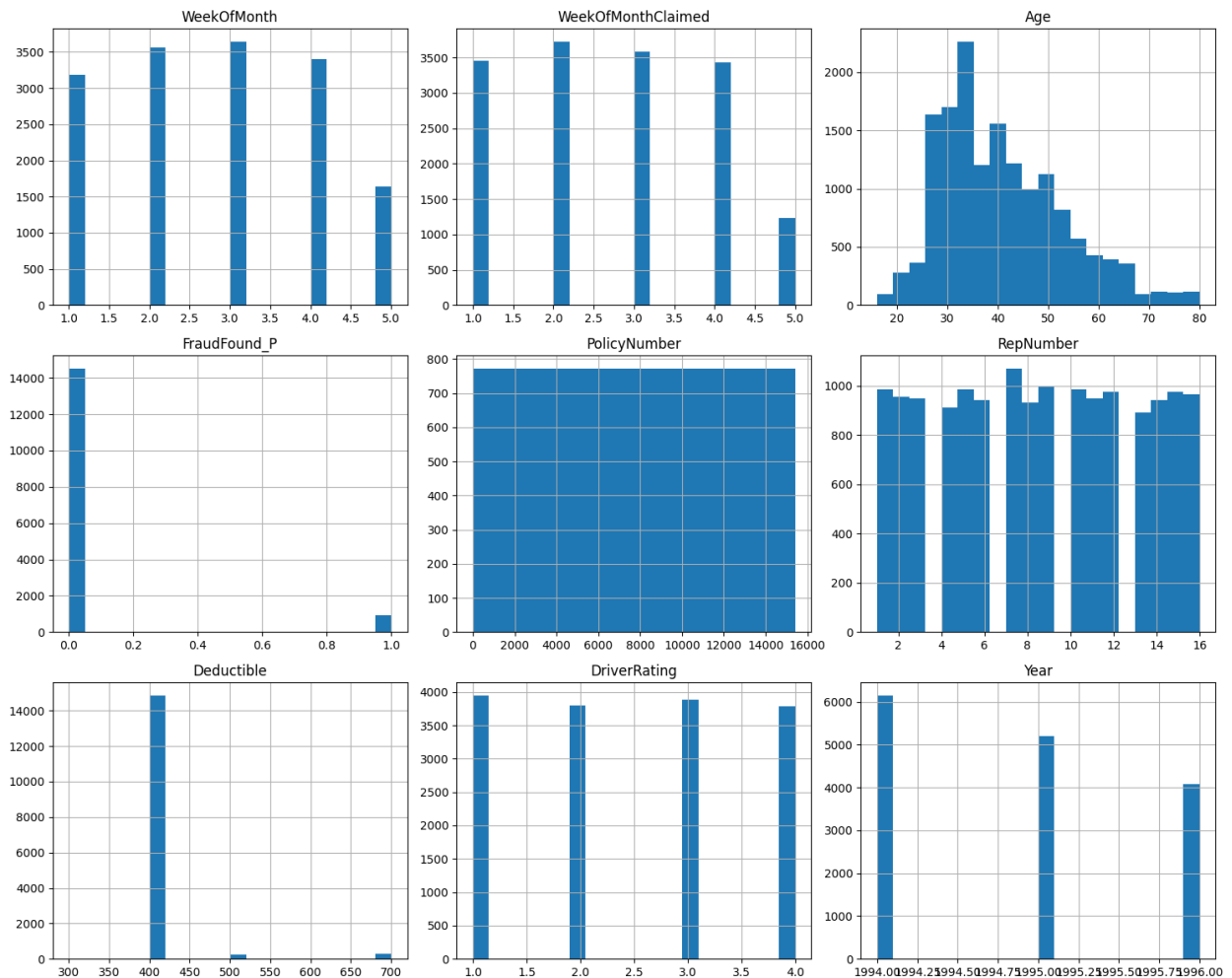
```
df1[numeric_columns].describe()
```

	WeekOfMonth	WeekOfMonthClaimed	Age	FraudFound_P \
count	15420.000000	15420.000000	15420.000000	15420.000000
mean	2.788586	2.693969	40.700331	0.059857
std	1.287585	1.259115	12.181090	0.237230
min	1.000000	1.000000	16.000000	0.000000
25%	2.000000	2.000000	31.000000	0.000000
50%	3.000000	3.000000	39.000000	0.000000
75%	4.000000	4.000000	48.000000	0.000000
max	5.000000	5.000000	80.000000	1.000000

	PolicyNumber	RepNumber	Deductible	DriverRating
Year				
count	15420.000000	15420.000000	15420.000000	15420.000000
15420.000000				
mean	7710.500000	8.483268	407.704280	2.487808
1994.866472				
std	4451.514911	4.599948	43.950998	1.119453
0.803313				
min	1.000000	1.000000	300.000000	1.000000
1994.000000				
25%	3855.750000	5.000000	400.000000	1.000000
1994.000000				
50%	7710.500000	8.000000	400.000000	2.000000
1995.000000				
75%	11565.250000	12.000000	400.000000	3.000000
1996.000000				
max	15420.000000	16.000000	700.000000	4.000000
1996.000000				

```
df1[numeric_columns].hist(  
    figsize=(15, 12),  
    bins=20  
)
```

```
plt.tight_layout()
plt.show()
```



```
for column in categorical_columns:
    print("\n", column)
    print(df1[column].unique())
```

```
Month
<StringArray>
['Dec', 'Jan', 'Oct', 'Jun', 'Feb', 'Nov', 'Apr', 'Mar', 'Aug', 'Jul',
 'May',
 'Sep']
Length: 12, dtype: str
```

```
DayOfWeek
<StringArray>
['Wednesday', 'Friday', 'Saturday', 'Monday', 'Tuesday', 'Sunday',
```

```
'Thursday']
Length: 7, dtype: str

Make
<StringArray>
[ 'Honda', 'Toyota', 'Ford', 'Mazda', 'Chevrolet',
  'Pontiac',
  'Accura', 'Dodge', 'Mercury', 'Jaguar', 'Nisson',
  'VW',
  'Saab', 'Saturn', 'Porche', 'BMW', 'Mecedes',
  'Ferrari',
  'Lexus']
Length: 19, dtype: str

AccidentArea
<StringArray>
['Urban', 'Rural']
Length: 2, dtype: str

DayOfWeekClaimed
<StringArray>
[ 'Tuesday', 'Monday', 'Thursday', 'Friday', 'Wednesday',
  'Saturday',
  'Sunday', '0']
Length: 8, dtype: str

MonthClaimed
<StringArray>
['Jan', 'Nov', 'Jul', 'Feb', 'Mar', 'Dec', 'Apr', 'Aug', 'May', 'Jun',
  'Sep',
  'Oct', '0']
Length: 13, dtype: str

Sex
<StringArray>
['Female', 'Male']
Length: 2, dtype: str

MaritalStatus
<StringArray>
['Single', 'Married', 'Widow', 'Divorced']
Length: 4, dtype: str

Fault
<StringArray>
['Policy Holder', 'Third Party']
Length: 2, dtype: str

PolicyType
<StringArray>
```

```
[ 'Sport - Liability', 'Sport - Collision', 'Sedan - Liability',  
  'Utility - All Perils', 'Sedan - All Perils', 'Sedan - Collision',  
  'Utility - Collision', 'Utility - Liability', 'Sport - All Perils']  
Length: 9, dtype: str  
  
VehicleCategory  
<StringArray>  
['Sport', 'Utility', 'Sedan']  
Length: 3, dtype: str  
  
VehiclePrice  
<StringArray>  
['more than 69000', '20000 to 29000', '30000 to 39000', 'less than 20000',  
  '40000 to 59000', '60000 to 69000']  
Length: 6, dtype: str  
  
Days_Policy_Accident  
<StringArray>  
['more than 30', '15 to 30', 'none', '1 to 7', '8 to 15']  
Length: 5, dtype: str  
  
Days_Policy_Claim  
<StringArray>  
['more than 30', '15 to 30', '8 to 15', 'none']  
Length: 4, dtype: str  
  
PastNumberOfClaims  
<StringArray>  
['none', '1', '2 to 4', 'more than 4']  
Length: 4, dtype: str  
  
AgeOfVehicle  
<StringArray>  
[ '3 years', '6 years', '7 years', 'more than 7', '5 years',  
  'new', '4 years', '2 years']  
Length: 8, dtype: str  
  
AgeOfPolicyHolder  
<StringArray>  
['26 to 30', '31 to 35', '41 to 50', '51 to 65', '21 to 25', '36 to 40',  
  '16 to 17', 'over 65', '18 to 20']  
Length: 9, dtype: str  
  
PoliceReportFiled
```

```

<StringArray>
['No', 'Yes']
Length: 2, dtype: str

    WitnessPresent
<StringArray>
['No', 'Yes']
Length: 2, dtype: str

    AgentType
<StringArray>
['External', 'Internal']
Length: 2, dtype: str

    NumberOfSupplements
<StringArray>
['none', 'more than 5', '3 to 5', '1 to 2']
Length: 4, dtype: str

    AddressChange_Claim
<StringArray>
['1 year', 'no change', '4 to 8 years', '2 to 3 years', 'under 6
months']
Length: 5, dtype: str

    NumberOfCars
<StringArray>
['3 to 4', '1 vehicle', '2 vehicles', '5 to 8', 'more than 8']
Length: 5, dtype: str

    BasePolicy
<StringArray>
['Liability', 'Collision', 'All Perils']
Length: 3, dtype: str

for column in categorical_columns:

    if column != 'FraudFound_P':

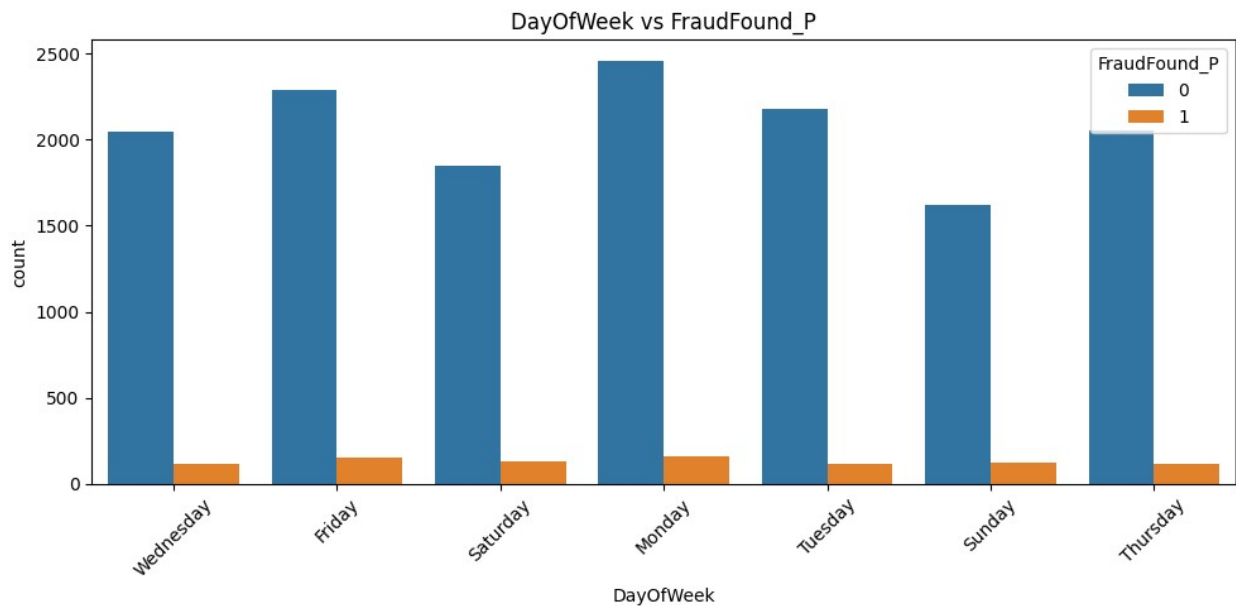
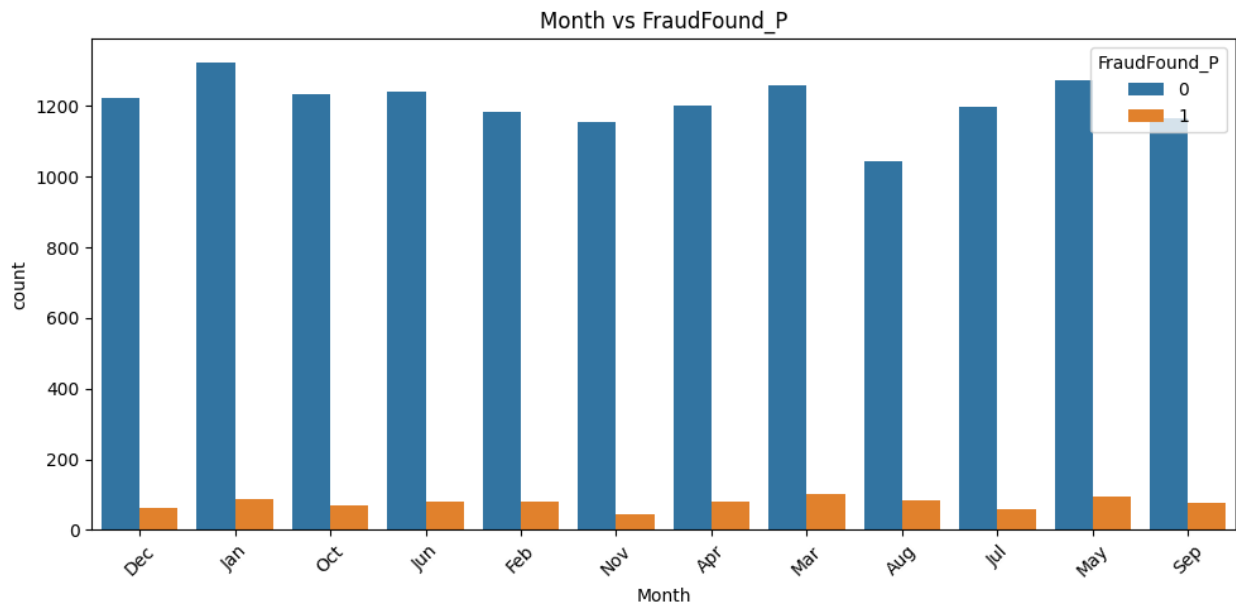
        plt.figure(figsize=(10, 5))

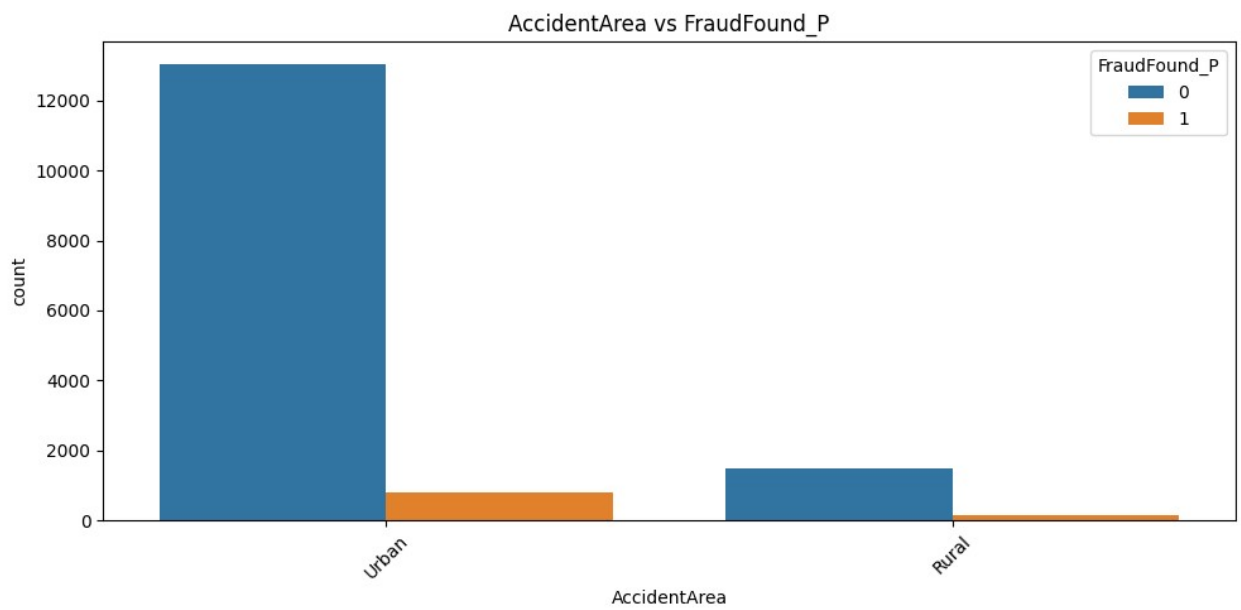
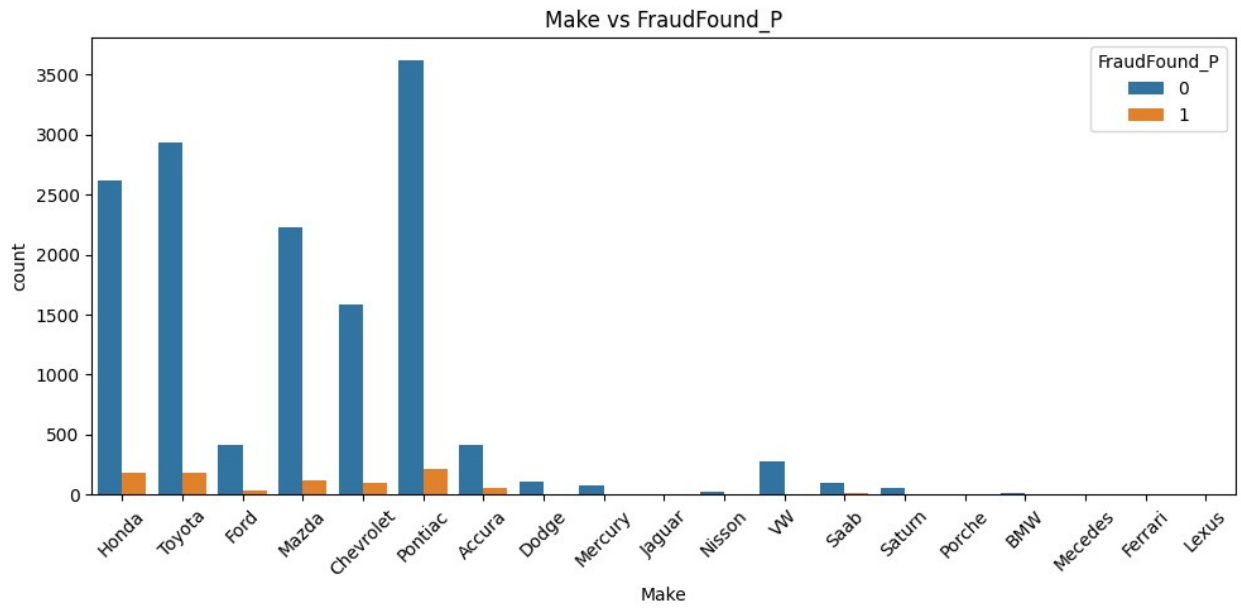
        sns.countplot(
            data=df,
            x=column,
            hue='FraudFound_P'
        )

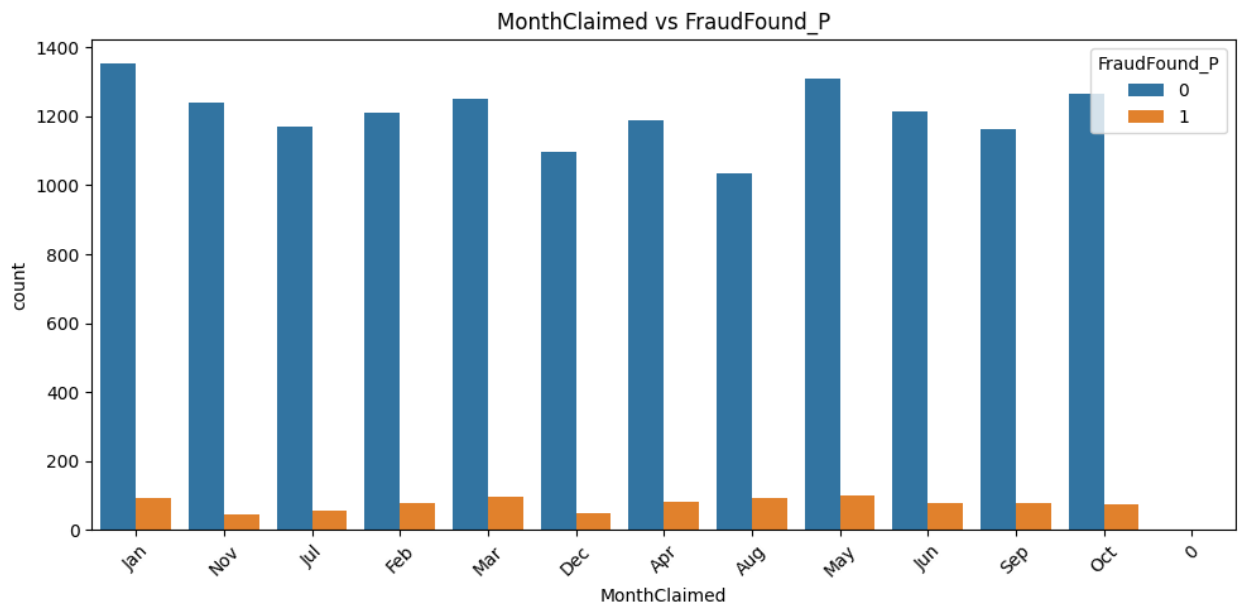
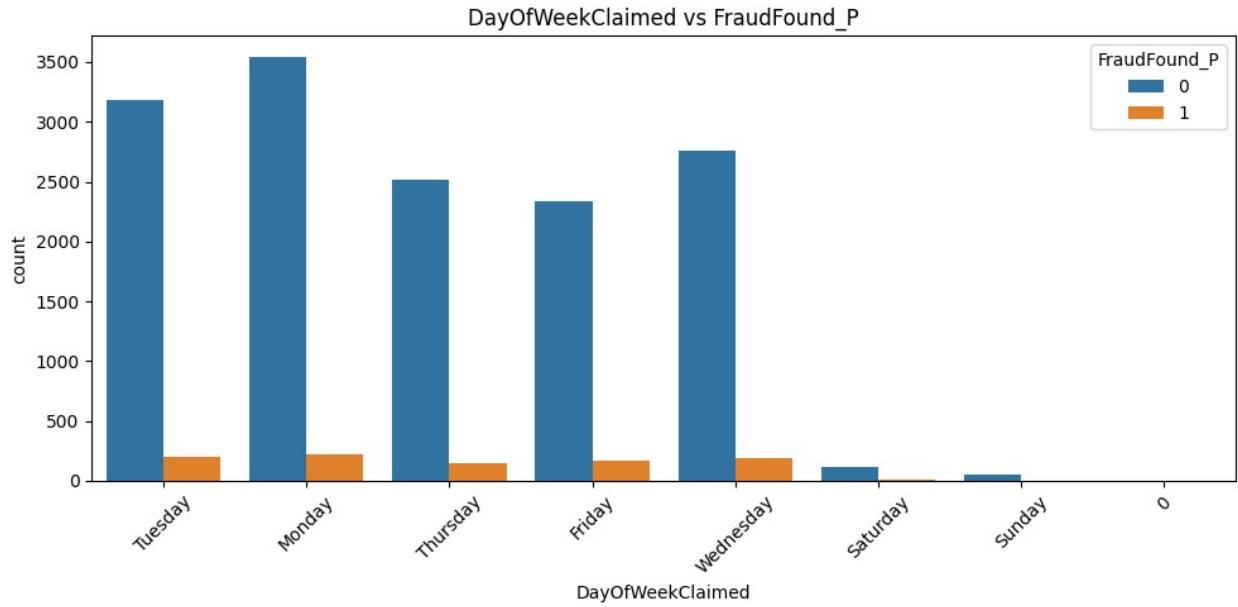
        plt.title(f'{column} vs FraudFound_P')
        plt.xticks(rotation=45)

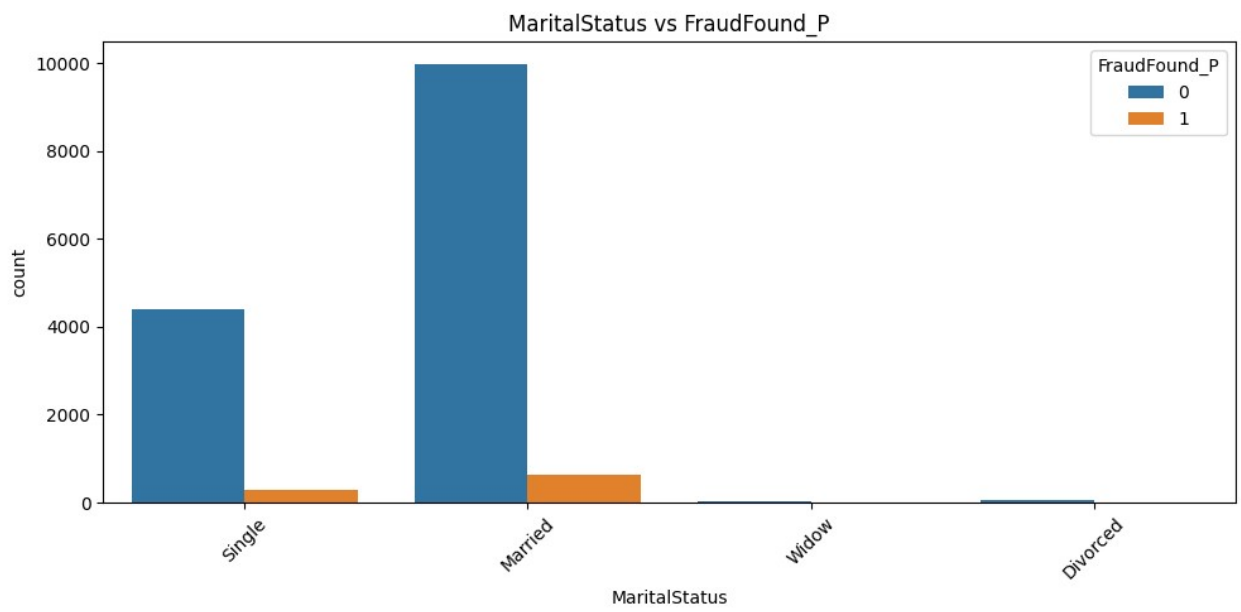
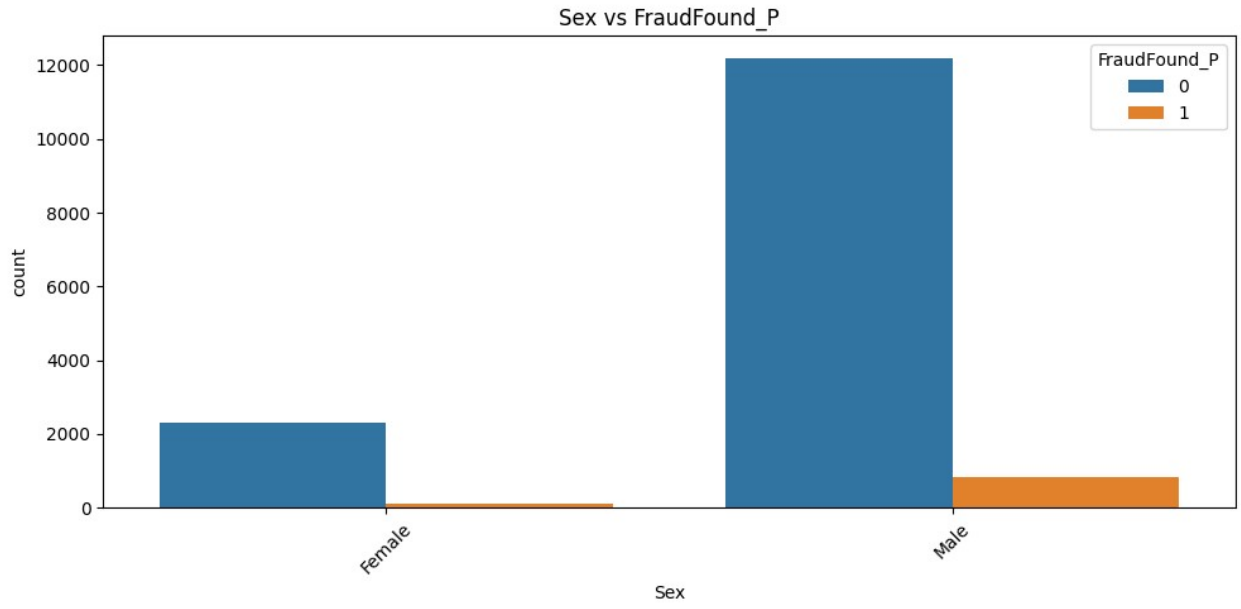
```

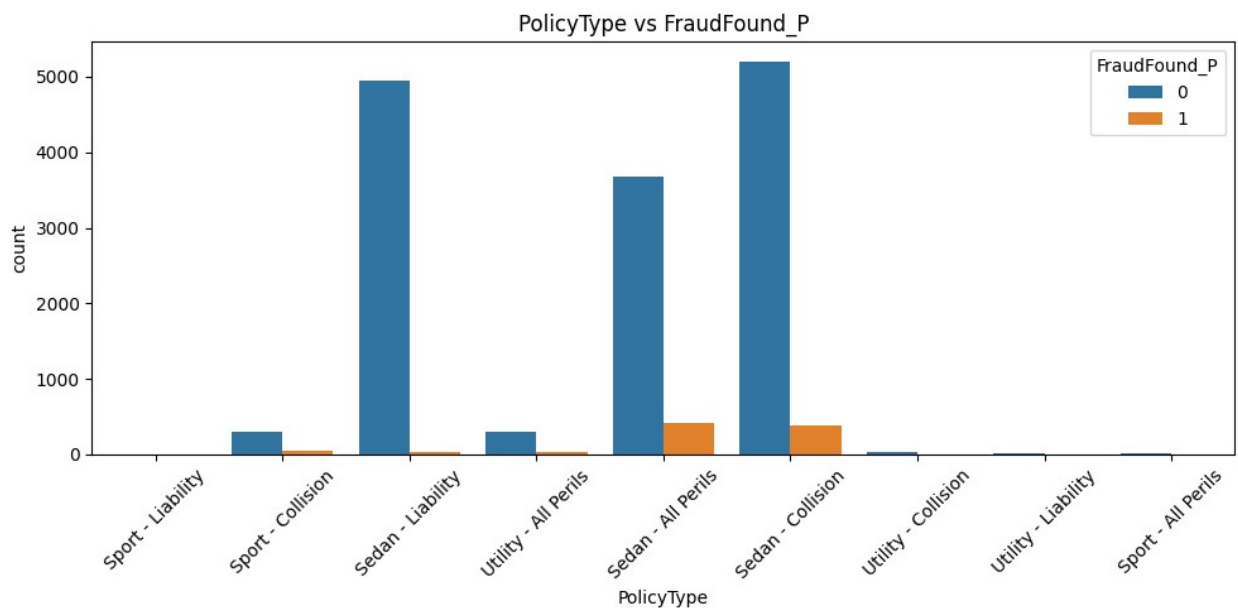
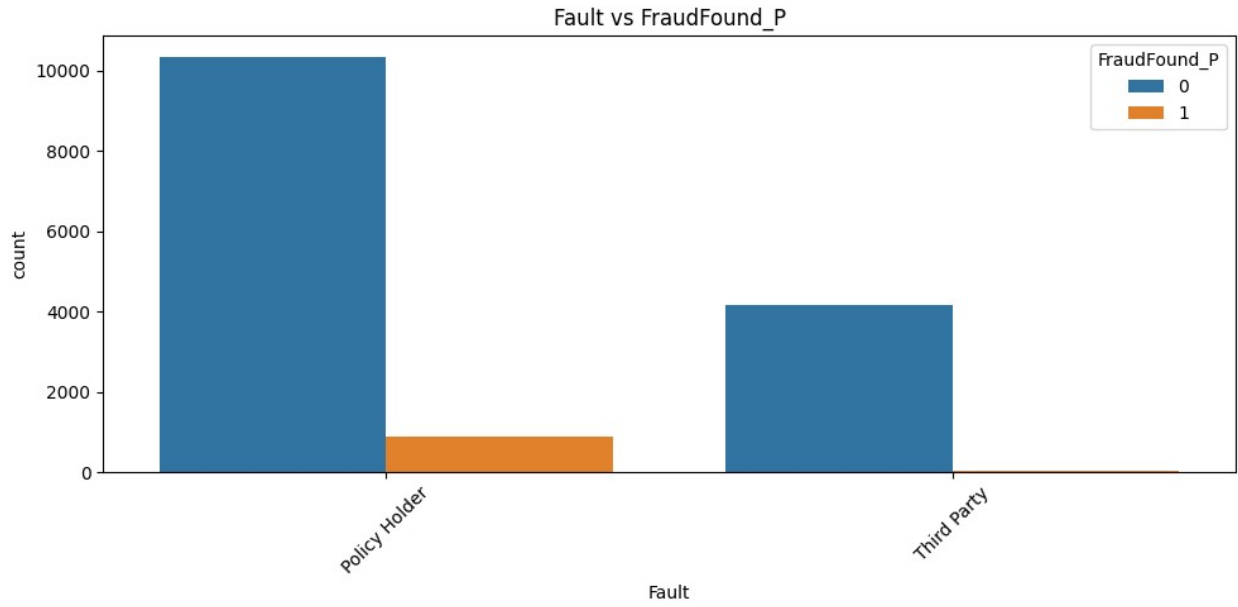
```
plt.tight_layout()  
plt.show()
```

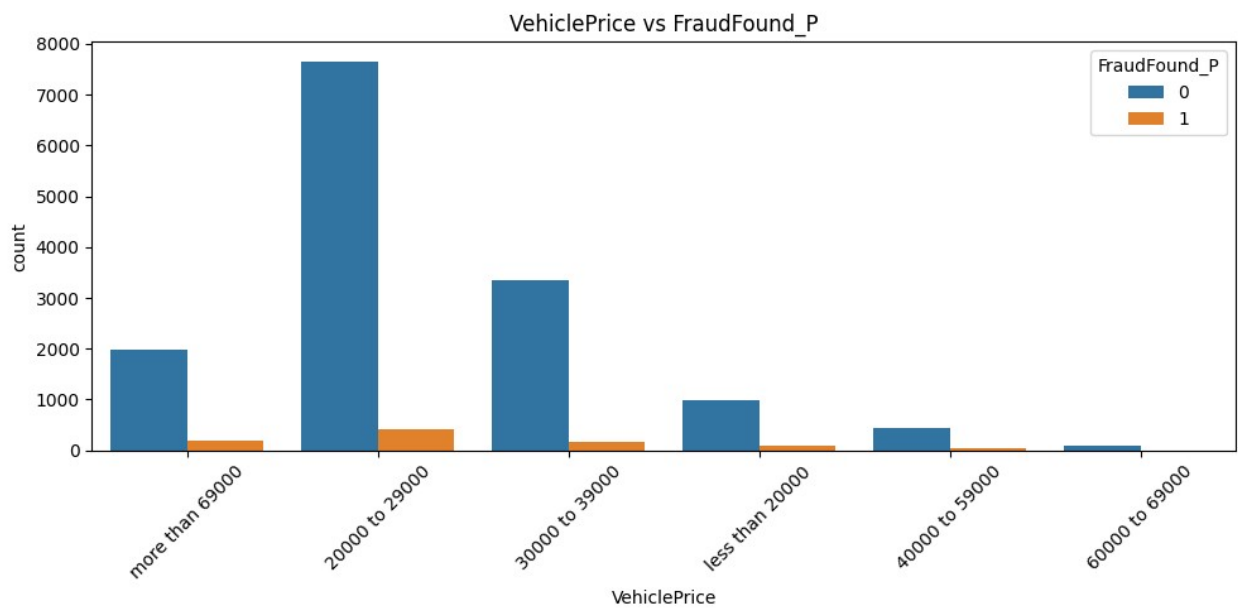
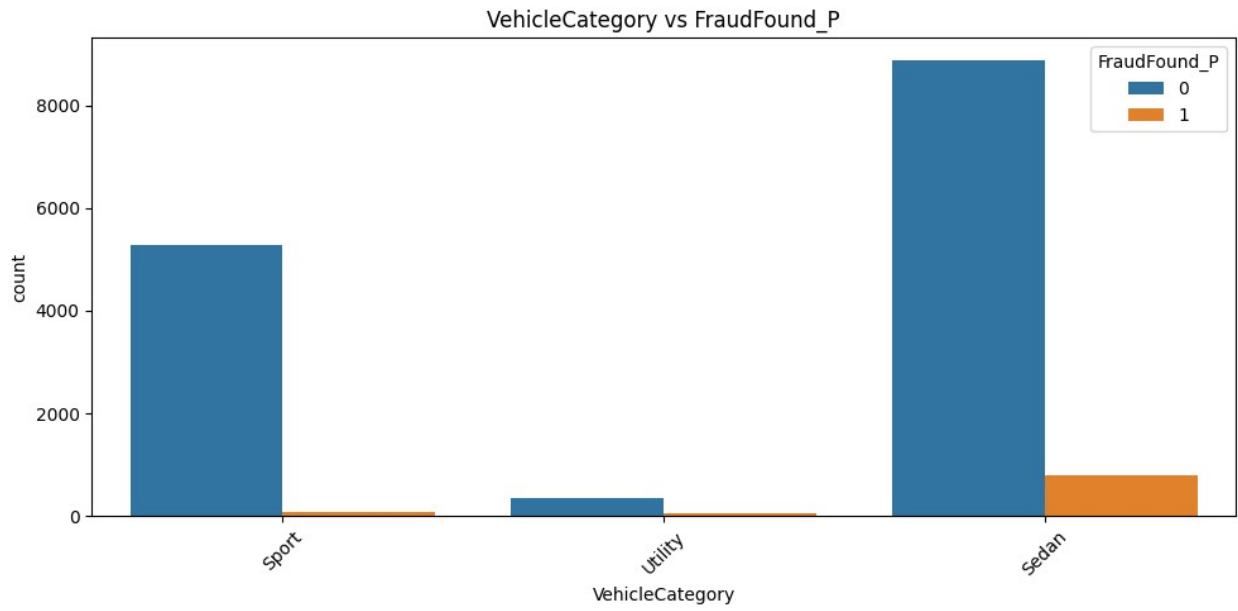


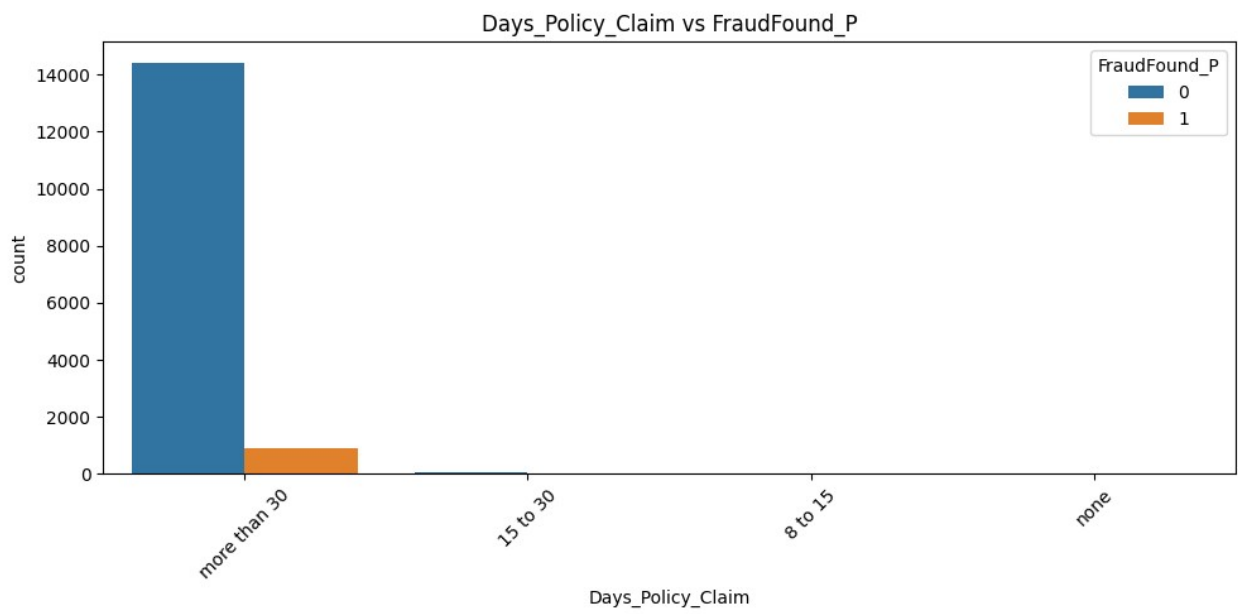
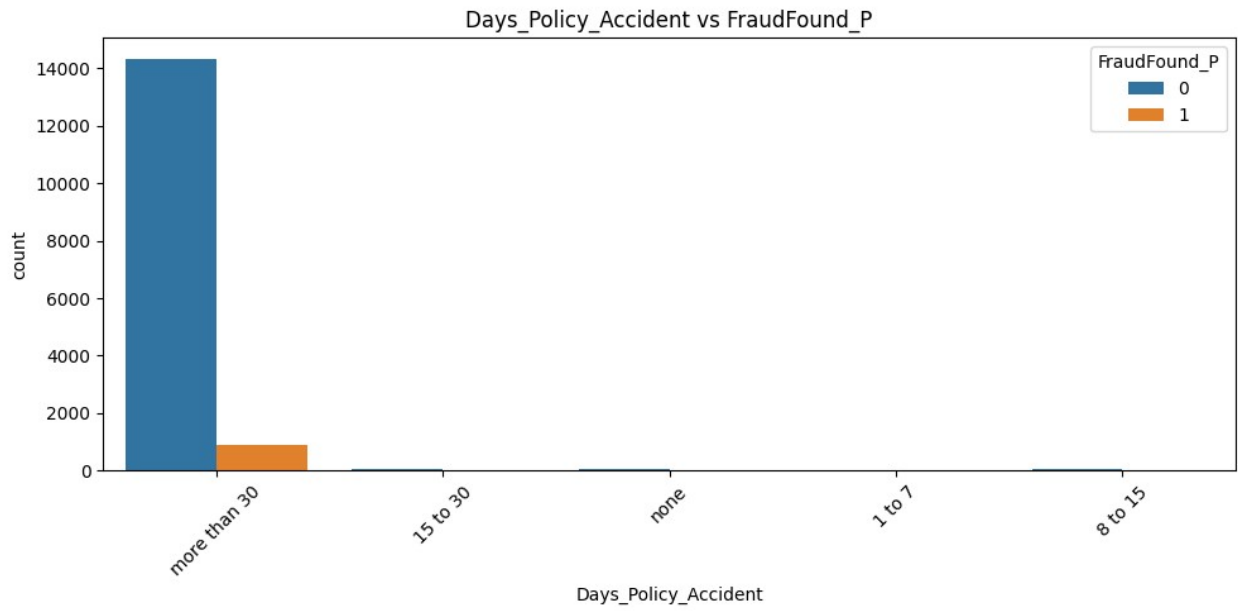


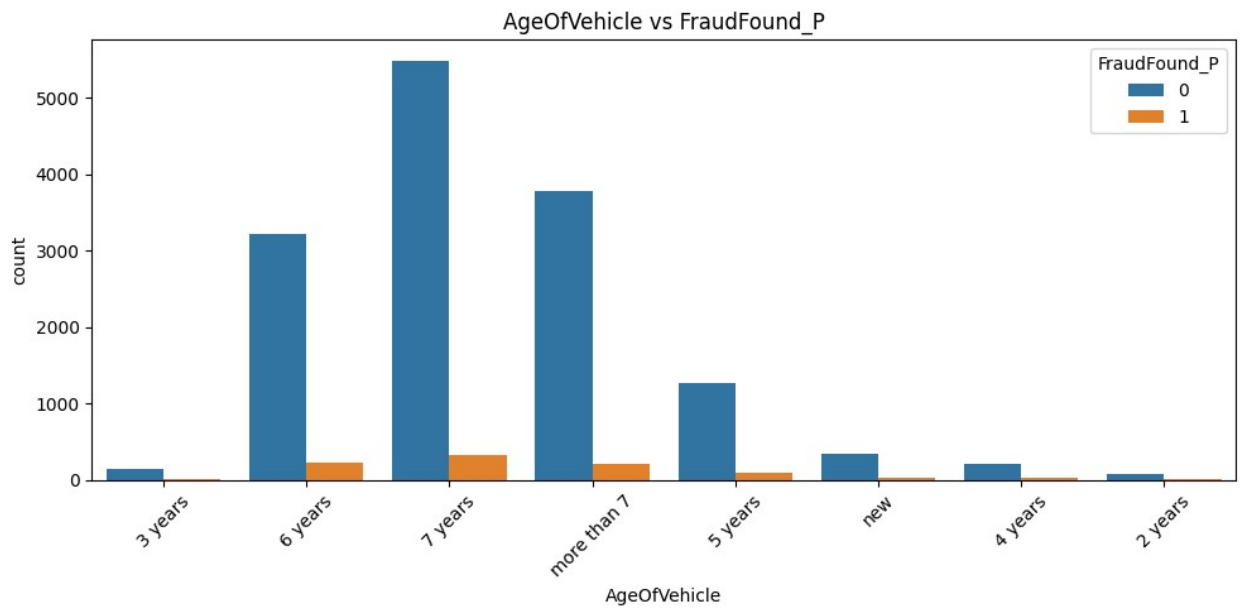
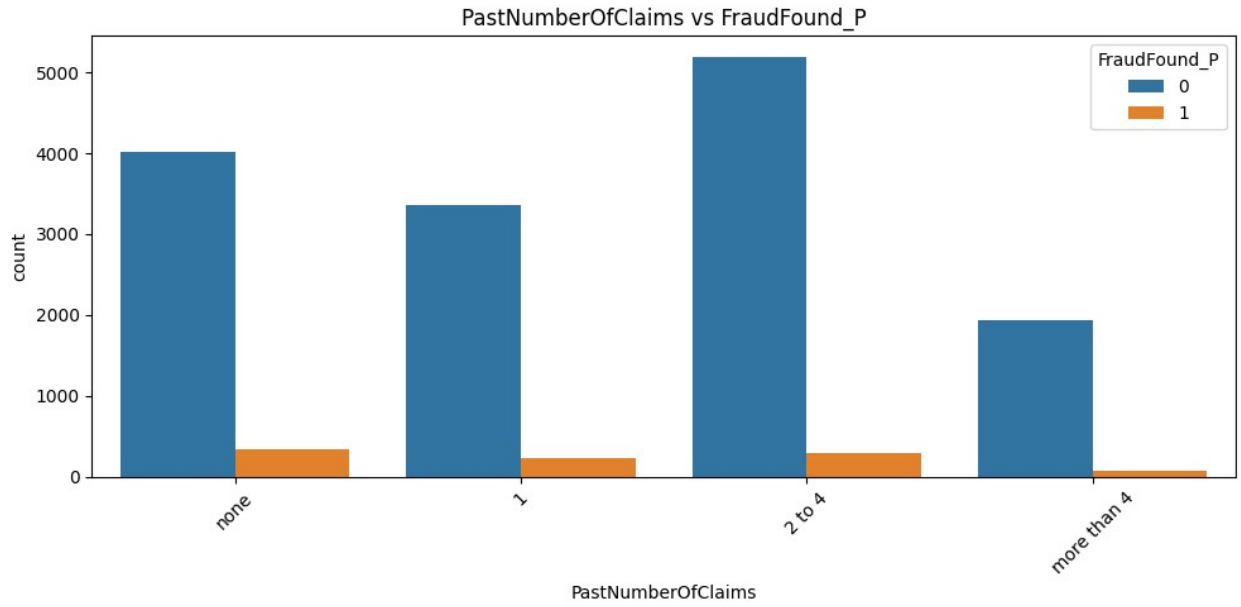


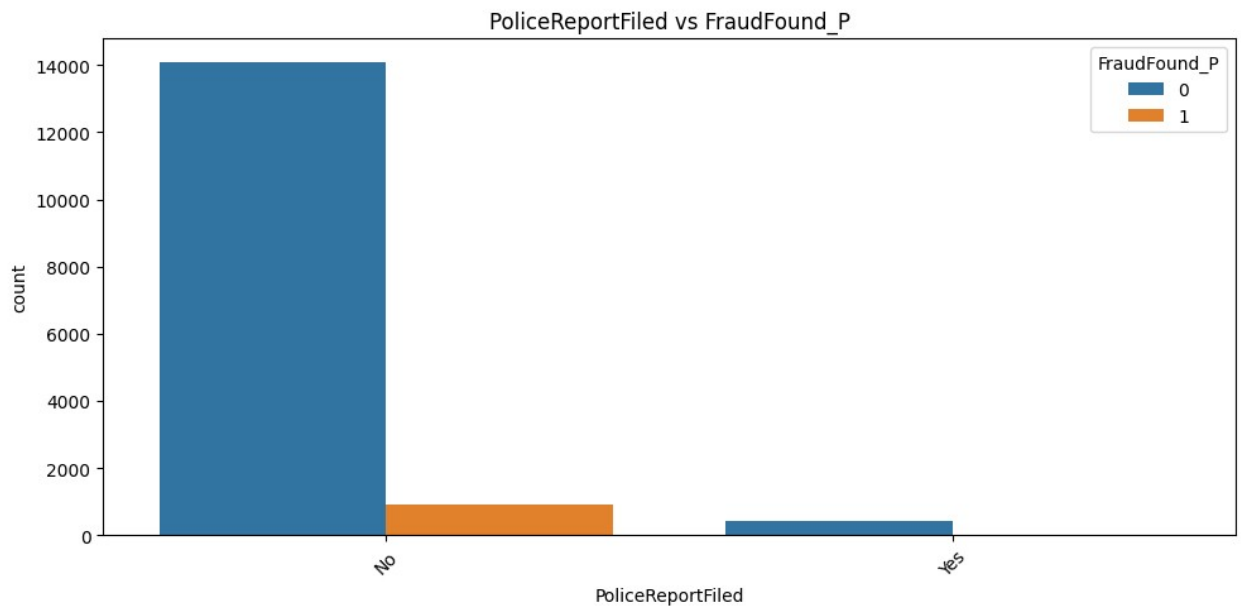
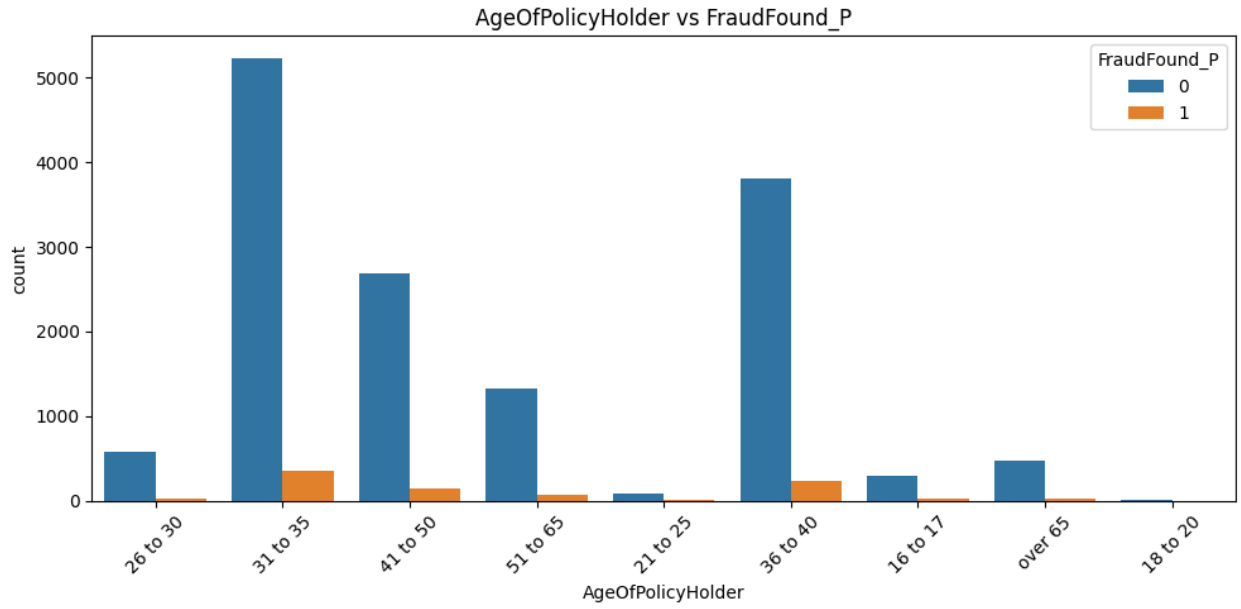


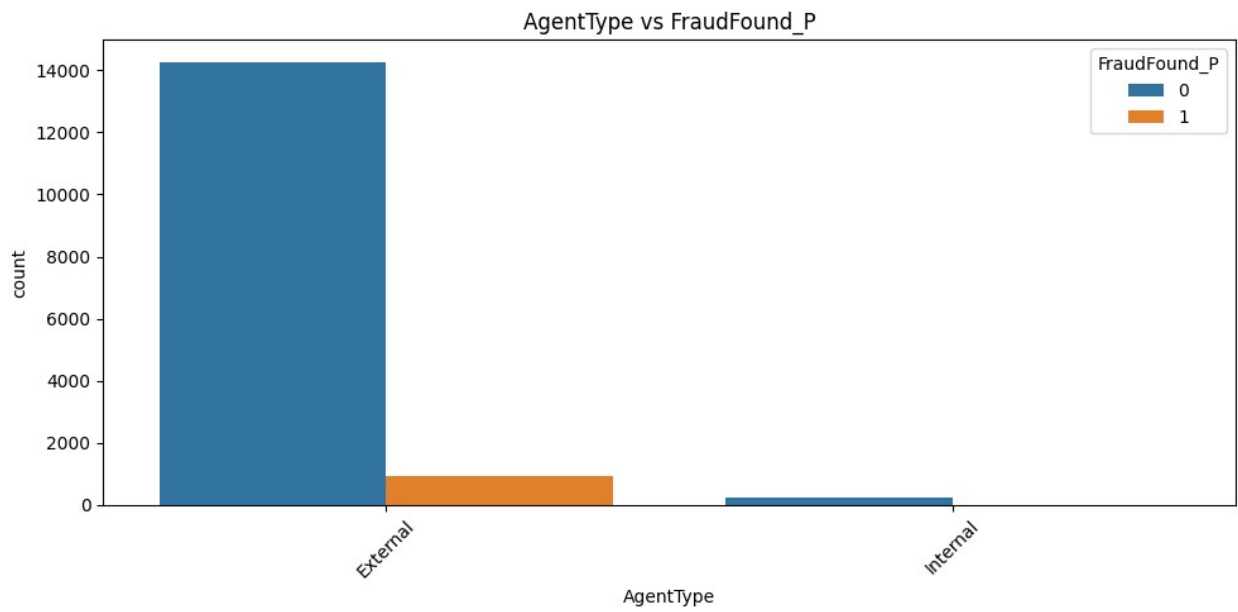
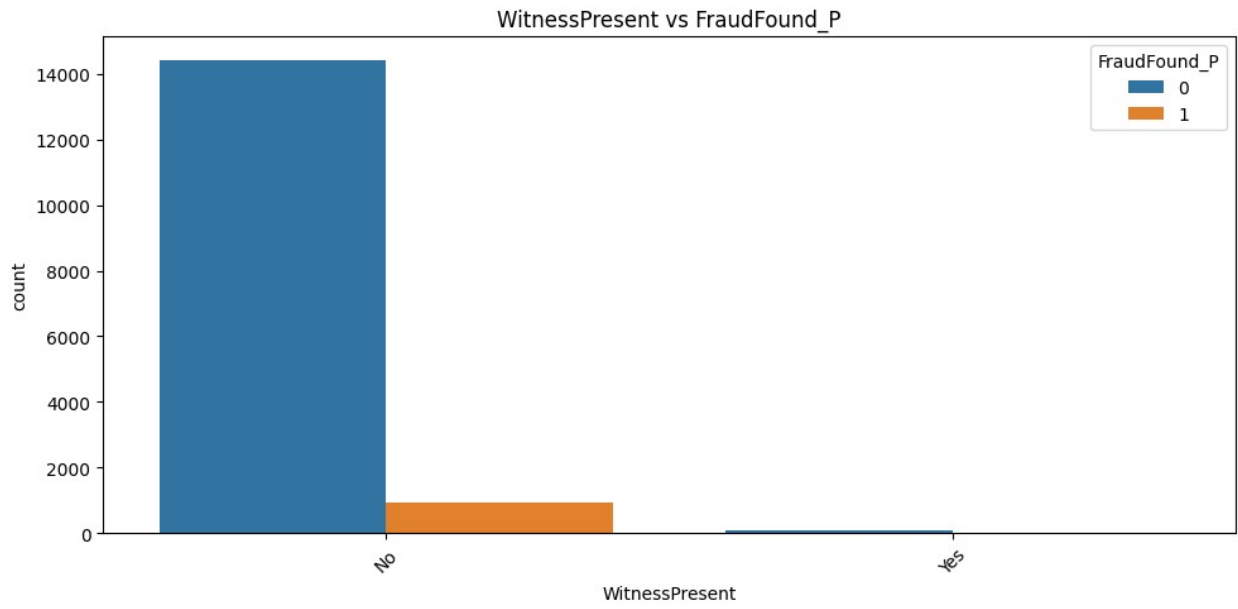


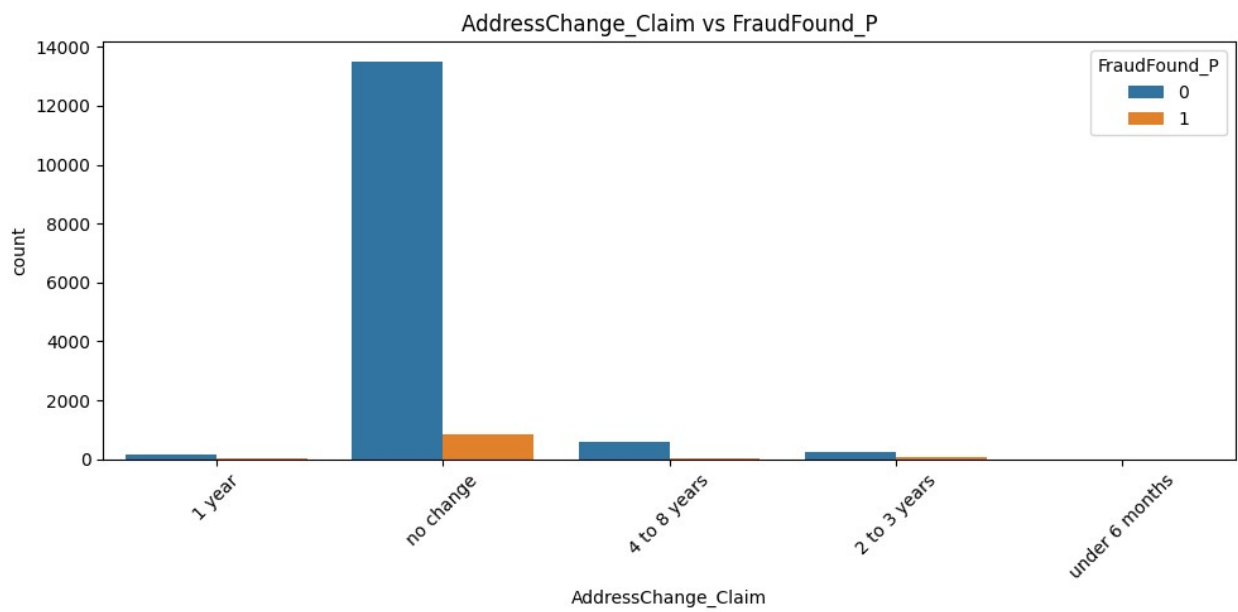
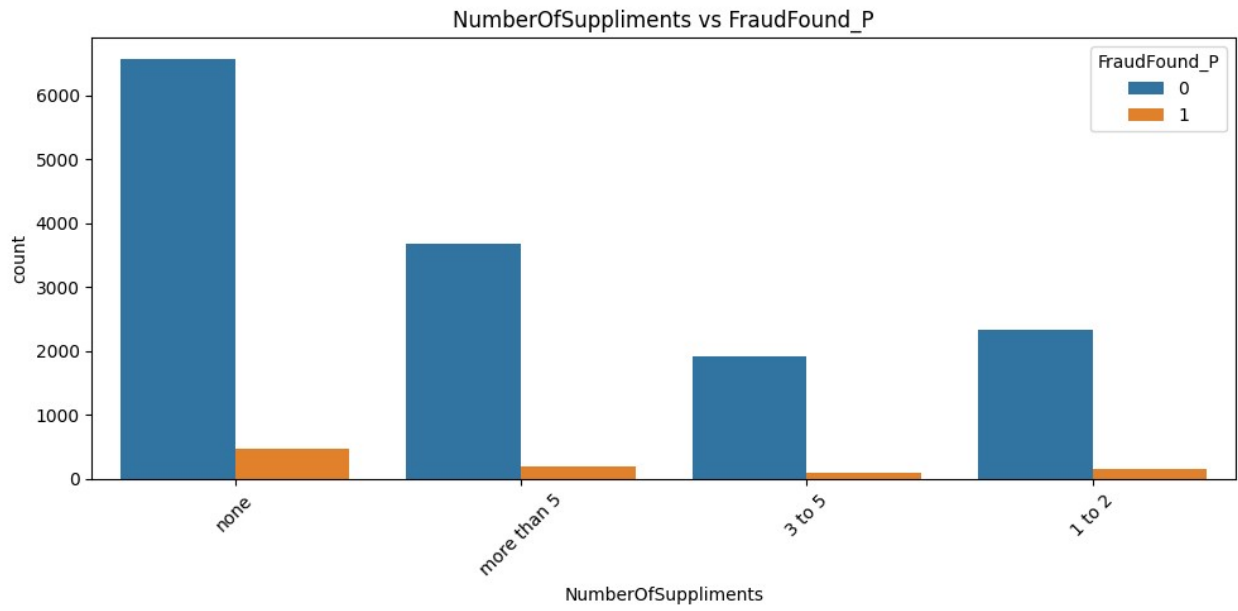


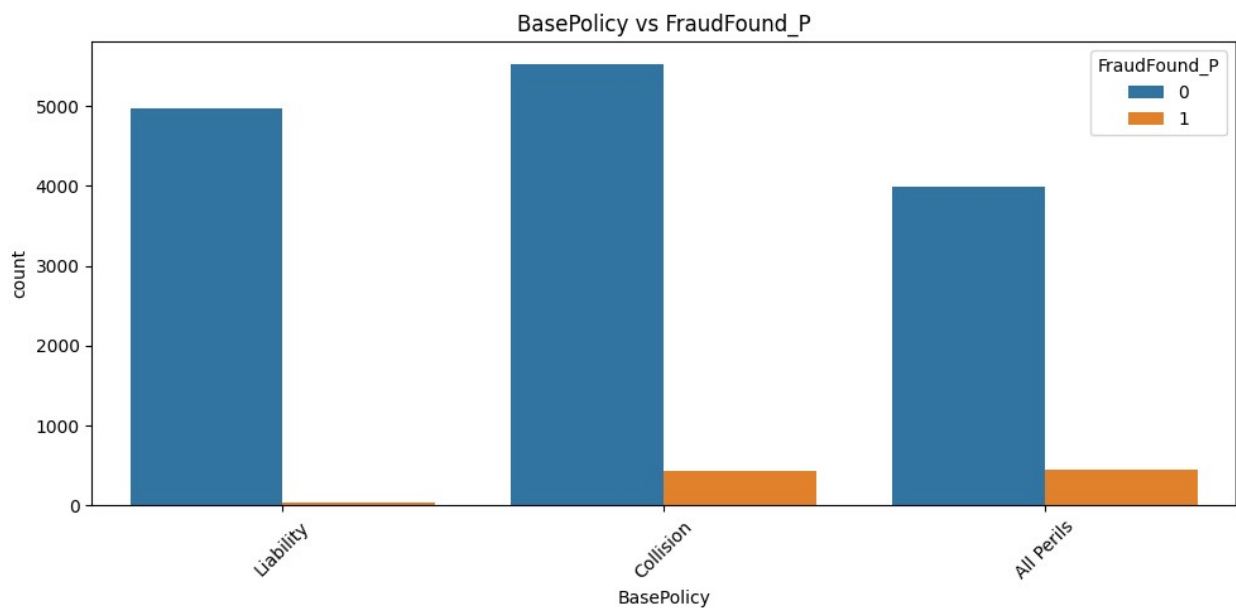
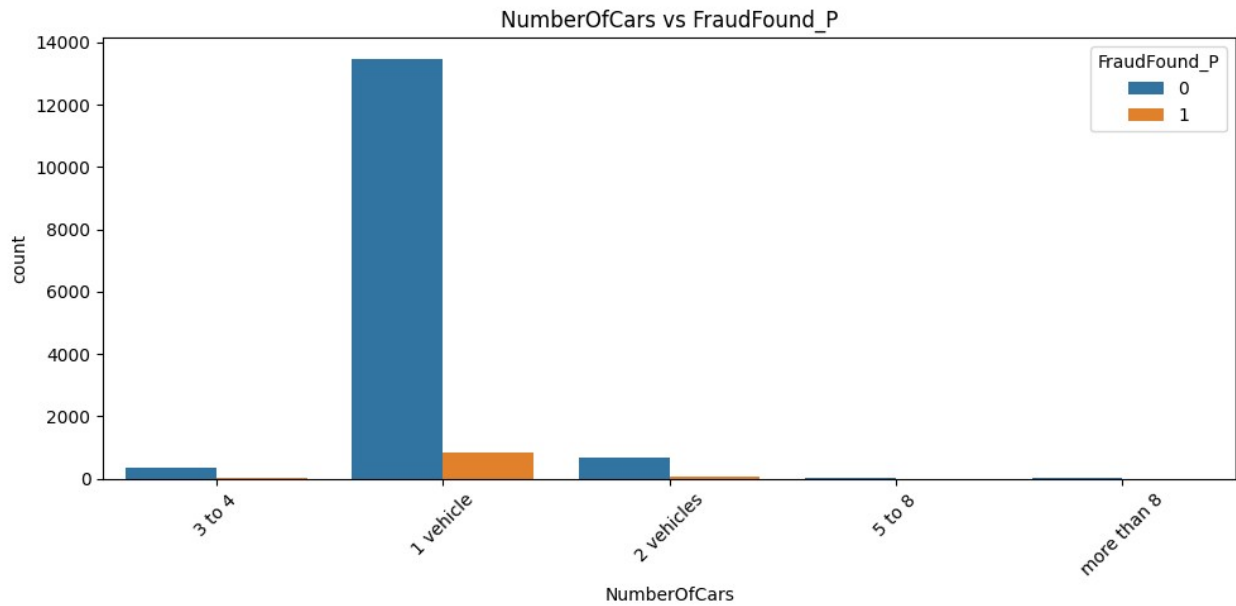












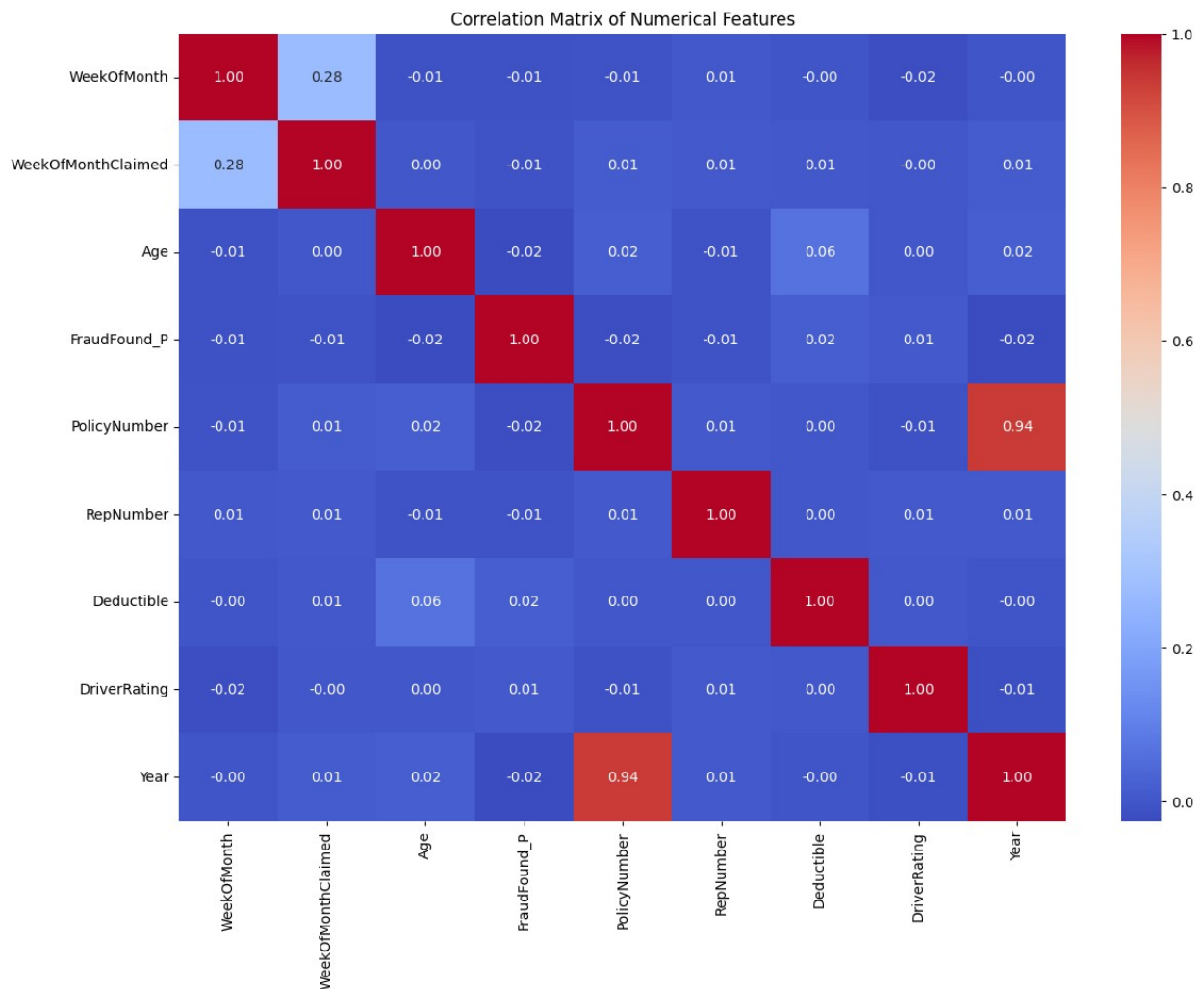
1. Correlation Matrix

```
correlation_matrix = df1[numeric_columns].corr()

plt.figure(figsize=(14, 10))

sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap='coolwarm',
    fmt='.2f'
)
```

```
plt.title('Correlation Matrix of Numerical Features')
plt.show()
```



1. Encode categorical variables

```
binary_cols = [col for col in df1.columns if df1[col].dtype == 'str'
and df1[col].nunique() == 2]
print(binary_cols)

['AccidentArea', 'Sex', 'Fault', 'PoliceReportFiled',
'WitnessPresent', 'AgentType']

binary_cols = [col for col in df1.columns if df1[col].dtype == 'str'
and df1[col].nunique() == 2]
for column in binary_cols:
    print(f"{column}: {df1[column].unique()}")
```

```

AccidentArea: <StringArray>
['Urban', 'Rural']
Length: 2, dtype: str
Sex: <StringArray>
['Female', 'Male']
Length: 2, dtype: str
Fault: <StringArray>
['Policy Holder', 'Third Party']
Length: 2, dtype: str
PoliceReportFiled: <StringArray>
['No', 'Yes']
Length: 2, dtype: str
WitnessPresent: <StringArray>
['No', 'Yes']
Length: 2, dtype: str
AgentType: <StringArray>
['External', 'Internal']
Length: 2, dtype: str

from sklearn.preprocessing import LabelEncoder
# To encode/transform categorical column values to numerical values
le = LabelEncoder()

for col in binary_cols:
    # fit() - train label encoder with input samples
    # transform() - apply changes and transform existing dataframe
    # samples.
    df1[col] = le.fit_transform(df1[col])

non_binary_cols = [col for col in df1.columns if df1[col].dtype ==
'<str' and df1[col].nunique() > 2]

print(non_binary_cols)

['Month', 'DayOfWeek', 'Make', 'DayOfWeekClaimed', 'MonthClaimed',
'MaritalStatus', 'PolicyType', 'VehicleCategory', 'VehiclePrice',
'Days_Policy_Accident', 'Days_Policy_Claim', 'PastNumberOfClaims',
'AgeOfVehicle', 'AgeOfPolicyHolder', 'NumberOfSupplements',
'AddressChange_Claim', 'NumberOfCars', 'BasePolicy']

df['AgeOfVehicle'].unique()

<StringArray>
[      '3 years',      '6 years',      '7 years', 'more than 7',      '5
years',
      'new',      '4 years',      '2 years']
Length: 8, dtype: str

df1['AgeOfVehicle'] = df1['AgeOfVehicle'].str.replace(' years',
'').str.replace('more than', '').str.replace('new', '0')

```

```

df1['AgeOfVehicle'].unique()

<StringArray>
['3', '6', '7', '7', '5', '0', '4', '2']
Length: 8, dtype: str

df1['AgeOfVehicle'] = df1['AgeOfVehicle'].astype('float')

df1['AgeOfPolicyHolder'].unique()

<StringArray>
['26 to 30', '31 to 35', '41 to 50', '51 to 65', '21 to 25', '36 to 40',
 '16 to 17', 'over 65', '18 to 20']
Length: 9, dtype: str

for col in non_binary_cols:
    # fit() - train label encoder with input samples
    # transform() - apply changes and transform existing dataframe
    samples.
    df1[col] = le.fit_transform(df1[col])

df1.info()

<class 'pandas.DataFrame'>
RangeIndex: 15420 entries, 0 to 15419
Data columns (total 33 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Month                                15420 non-null  int64
1   WeekOfMonth                          15420 non-null  int64
2   DayOfWeek                            15420 non-null  int64
3   Make                                 15420 non-null  int64
4   AccidentArea                         15420 non-null  int64
5   DayOfWeekClaimed                     15420 non-null  int64
6   MonthClaimed                         15420 non-null  int64
7   WeekOfMonthClaimed                   15420 non-null  int64
8   Sex                                  15420 non-null  int64
9   MaritalStatus                       15420 non-null  int64
10  Age                                  15420 non-null  float64
11  Fault                               15420 non-null  int64
12  PolicyType                           15420 non-null  int64
13  VehicleCategory                      15420 non-null  int64
14  VehiclePrice                         15420 non-null  int64
15  FraudFound_P                         15420 non-null  int64
16  PolicyNumber                         15420 non-null  int64
17  RepNumber                            15420 non-null  int64
18  Deductible                           15420 non-null  int64
19  DriverRating                         15420 non-null  int64
20  Days_Policy_Accident                 15420 non-null  int64
21  Days_Policy_Claim                    15420 non-null  int64

```

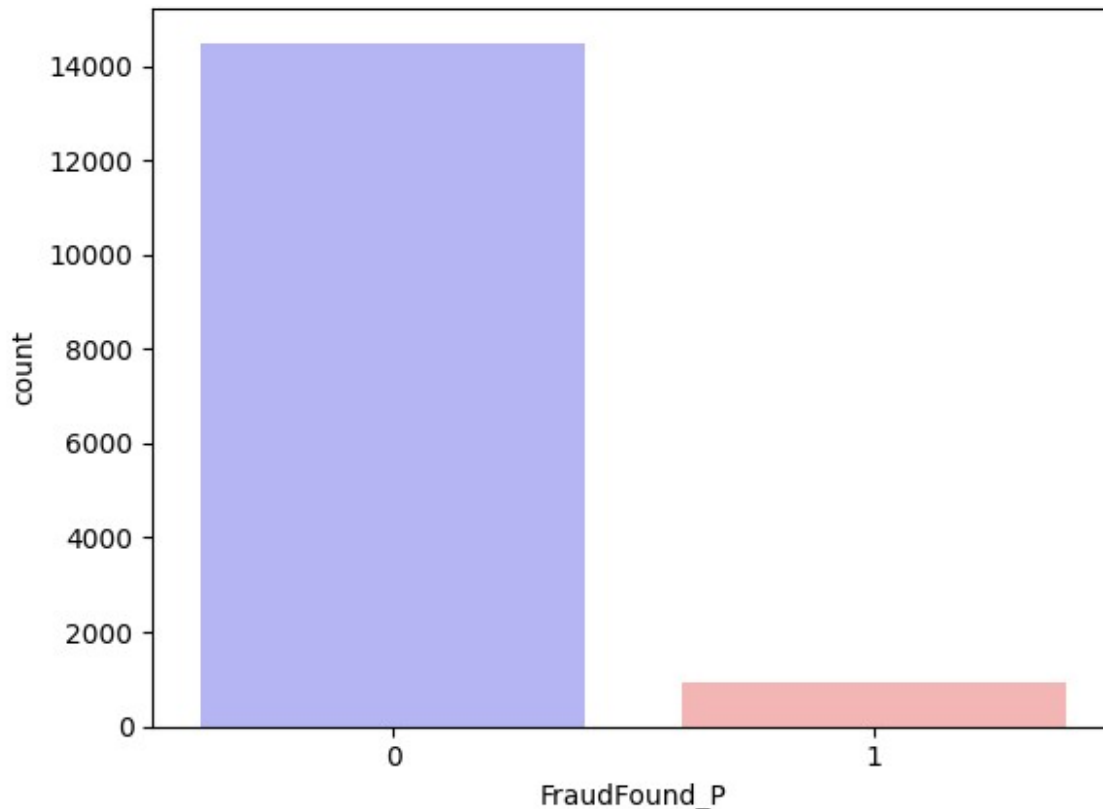
```
22  PastNumberOfClaims      15420 non-null  int64
23  AgeOfVehicle            15420 non-null  int64
24  AgeOfPolicyHolder       15420 non-null  int64
25  PoliceReportFiled       15420 non-null  int64
26  WitnessPresent          15420 non-null  int64
27  AgentType               15420 non-null  int64
28  NumberOfSupplements     15420 non-null  int64
29  AddressChange_Claim     15420 non-null  int64
30  NumberOfCars            15420 non-null  int64
31  Year                    15420 non-null  int64
32  BasePolicy              15420 non-null  int64
dtypes: float64(1), int64(32)
memory usage: 3.9 MB
```

1. Split DataSet Into Train and Test Sets

```
data = df1.copy()
print(data.shape)
(15420, 33)
X = data.drop('FraudFound_P', axis=1)
Y = data['FraudFound_P']
print("X shape:", X.shape)
print("Y shape:", Y.shape)
X shape: (15420, 32)
Y shape: (15420,)
```

1. Applying SMOTE

```
sns.countplot(x = 'FraudFound_P', data = df1, palette = 'bwr')
<Axes: xlabel='FraudFound_P', ylabel='count'>
```



```
!pip install imbalanced-learn
```

```
Collecting imbalanced-learn
```

```
  Downloading imbalanced_learn-0.14.2-py3-none-any.whl.metadata (8.9 kB)
```

```
Requirement already satisfied: numpy<3,>=1.25.2 in c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (2.4.6)
```

```
Requirement already satisfied: scipy<2,>=1.11.4 in c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (1.15.2)
```

```
Requirement already satisfied: scikit-learn<2,>=1.4.2 in c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (1.9.1)
```

```
Collecting sklearn-compat<0.2,>=0.1.6 (from imbalanced-learn)
```

```
  Downloading sklearn_compat-0.1.6-py3-none-any.whl.metadata (22 kB)
```

```
Requirement already satisfied: joblib<2,>=1.2.0 in c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (1.6.0)
```

```
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages (from imbalanced-learn) (3.7.0)
```

```
Requirement already satisfied: cloudpickle>=3.0 in c:\users\shiva\appdata\local\programs\python\python312\lib\site-packages (from
```

```
joblib<2,>=1.2.0->imbalanced-learn) (3.1.2)
Requirement already satisfied: narwhals>=2.0.1 in c:\users\shiva\
appdata\local\programs\python\python312\lib\site-packages (from
scikit-learn<2,>=1.4.2->imbalanced-learn) (2.26.0)
Downloading imbalanced_learn-0.14.2-py3-none-any.whl (236 kB)
Downloading sklearn_compat-0.1.6-py3-none-any.whl (22 kB)
Installing collected packages: sklearn-compat, imbalanced-learn
Successfully installed imbalanced-learn-0.14.2 sklearn-compat-0.1.6
```

```
[notice] A new release of pip is available: 25.0.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
from imblearn.over_sampling import SMOTE
smote = SMOTE()
transformed_feature, transformed_label = smote.fit_resample(X, Y)

from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test =
train_test_split(transformed_feature, transformed_label, test_size =
0.2,
random_state = 2)
```

1. Applying Logistic Regression

```
# import logistic regression & model evaluation
from sklearn.linear_model import LogisticRegression

# Hyperparameter - Performance tuning of ML Model
# solver : 'liblinear', 'lbfgs', 'newton-cg', 'newton-cholesky',
'sag', 'saga'
# max_iter : number of iterations
# C : inverse of regularization
# penalty : 'l1', 'l2' - avoids overfitting of regression model
logit_model = LogisticRegression(solver='lbfgs', max_iter = 10000, C =
1, penalty = 'l2')
logit_model.fit(X_train, Y_train)

LogisticRegression(C=1, max_iter=10000, penalty='l2')

logit_model.score(X_test, Y_test)

0.8649767201241594
```

1. Applying Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
# hyperparameter tuning is setup the users to improve performance of
ml model s
# criterion = 'entropy' measures information gain or 'gini' measures
impurity. This is by default
```

```

# max_depth = depth of decision tree
# ccp_alpha = Post pruning - First allow the tree to grow and then
cut off branches
# First grow decision tree then remove unnecessary branches that do
not improve model performance
dt_clf = DecisionTreeClassifier()
dt_clf.fit(X_train, Y_train)

DecisionTreeClassifier()

# training score
dt_clf.score(X_train, Y_train)

1.0

dt_clf.score(X_test, Y_test)

0.9161924469736161

dt_clf.get_depth()

32

path = dt_clf.cost_complexity_pruning_path(X_train, Y_train)
ccp_alphas = path.ccp_alphas
impurities = path.impurities

ccp_alphas

array([0.00000000e+00, 2.13567739e-05, 2.14280582e-05, 2.75442504e-05,
       2.75442504e-05, 2.78964787e-05, 2.80886032e-05, 2.83312290e-05,
       2.83481029e-05, 2.83996619e-05, 2.85081532e-05, 2.85081532e-05,
       2.85737457e-05, 2.85803556e-05, 2.86200391e-05, 2.86349796e-05,
       2.86724018e-05, 2.87254588e-05, 2.87310295e-05, 2.87418265e-05,
       2.88362104e-05, 3.39427285e-05, 3.39712422e-05, 3.43687369e-05,
       3.44192182e-05, 3.59272832e-05, 3.62086234e-05, 3.77236473e-05,
       3.77236473e-05, 3.82763748e-05, 3.83224354e-05, 3.88014658e-05,
       3.91933998e-05, 3.95200115e-05, 3.95200115e-05, 3.95200115e-05,
       3.95200115e-05, 3.95200115e-05, 3.95200115e-05, 3.97091025e-05,
       3.97199248e-05, 3.99581491e-05, 4.00332584e-05, 4.00332584e-05,
       4.00332584e-05, 4.00332584e-05, 4.02385572e-05, 4.02385572e-05,
       4.02385572e-05, 4.04181936e-05, 4.04181936e-05, 4.05766963e-05,
       4.06652892e-05, 4.07175876e-05, 4.07175876e-05, 4.07823214e-05,
       4.08436482e-05, 4.09571028e-05, 4.09571028e-05, 4.09571028e-05,
       4.10597522e-05, 4.10597522e-05, 4.10597522e-05, 4.10597522e-05,
       4.11530698e-05, 4.11530698e-05, 4.11530698e-05, 4.11530698e-05,
       4.12107072e-05, 4.12382729e-05, 4.12382729e-05, 4.13530361e-05,
       4.14545575e-05, 4.15159717e-05, 4.15187535e-05, 4.15729991e-05,
       4.15729991e-05, 4.15729991e-05, 4.15729991e-05, 4.16260936e-05,
       4.16260936e-05, 4.16541535e-05, 4.16756485e-05, 4.16756485e-05,
       4.16756485e-05, 4.16756485e-05, 4.17220063e-05, 4.17220063e-05,

```

[illegible]

7.05481197e-05, 7.07491115e-05, 7.08965055e-05, 7.11360207e-05,
7.13262240e-05, 7.14042074e-05, 7.18390302e-05, 7.18545664e-05,
7.18545664e-05, 7.18545664e-05, 7.18545664e-05, 7.18545664e-05,
7.18545664e-05, 7.18545664e-05, 7.18545664e-05, 7.18545664e-05,
7.18545664e-05, 7.18545664e-05, 7.18545664e-05, 7.18545664e-05,
7.18545664e-05, 7.24294029e-05, 7.24294029e-05, 7.27178691e-05,
7.28630515e-05, 7.29600212e-05, 7.36270699e-05, 7.36813774e-05,
7.39075540e-05, 7.39075540e-05, 7.39075540e-05, 7.39075540e-05,
7.39075540e-05, 7.39075540e-05, 7.39075540e-05, 7.39075540e-05,
7.39075540e-05, 7.39075540e-05, 7.39075540e-05, 7.39075540e-05,
7.39075540e-05, 7.39075540e-05, 7.39075540e-05, 7.39075540e-05,
7.40102033e-05, 7.41128527e-05, 7.47810069e-05, 7.47876791e-05,
7.52323451e-05, 7.54472947e-05, 7.54472947e-05, 7.54472947e-05,
7.54472947e-05, 7.57466887e-05, 7.66448708e-05, 7.66448708e-05,
7.66448708e-05, 7.66448708e-05, 7.66448708e-05, 7.66448708e-05,
7.68502574e-05, 7.69870354e-05, 7.72014036e-05, 7.72121437e-05,
7.72612456e-05, 7.74117914e-05, 7.74117914e-05, 7.75558139e-05,
7.75589391e-05, 7.76029317e-05, 7.76029317e-05, 7.76029317e-05,
7.76029317e-05, 7.76029317e-05, 7.76029317e-05, 7.77823167e-05,
7.79737939e-05, 7.79737939e-05, 7.79737939e-05, 7.81522446e-05,
7.82161463e-05, 7.83867997e-05, 7.83867997e-05, 7.83867997e-05,
7.83867997e-05, 7.83867997e-05, 7.83867997e-05, 7.85267761e-05,
7.90400230e-05, 7.90400230e-05, 7.90400230e-05, 7.90400230e-05,
7.90400230e-05, 7.94795317e-05, 7.95927504e-05, 7.95927504e-05,
7.97893086e-05, 7.98138414e-05, 7.98281511e-05, 7.98720232e-05,
7.99496558e-05, 8.00665168e-05, 8.01043692e-05, 8.03339558e-05,
8.03920434e-05, 8.04771143e-05, 8.04771143e-05, 8.04771143e-05,
8.04771143e-05, 8.04771143e-05, 8.04771143e-05, 8.05206625e-05,
8.08235008e-05, 8.08363872e-05, 8.08363872e-05, 8.08363872e-05,
8.08363872e-05, 8.11366181e-05, 8.11872065e-05, 8.12834461e-05, 8.13853068e-05,
8.14351752e-05, 8.14351752e-05, 8.15444379e-05, 8.18053165e-05,
8.18704681e-05, 8.19142056e-05, 8.20193587e-05, 8.20532790e-05,
8.21195044e-05, 8.21195044e-05, 8.21886417e-05, 8.22050456e-05,
8.23841365e-05, 8.26327513e-05, 8.26327513e-05, 8.27764604e-05,
8.28038336e-05, 8.28257321e-05, 8.29565555e-05, 8.30319433e-05,
8.31459982e-05, 8.32521872e-05, 8.33512970e-05, 8.34280020e-05,
8.34657401e-05, 8.35309334e-05, 8.36125863e-05, 8.36125863e-05,
8.37294789e-05, 8.38303274e-05, 8.41224192e-05, 8.48568212e-05,
8.48592479e-05, 8.49656190e-05, 8.52782773e-05, 8.55381751e-05,
8.56467851e-05, 8.56635235e-05, 8.59060615e-05, 8.62254796e-05,
8.66104148e-05, 8.84363894e-05, 8.84363894e-05, 8.84363894e-05,
8.88603435e-05, 8.88689587e-05, 8.88689587e-05, 9.00748314e-05,
9.03569873e-05, 9.05367536e-05, 9.07484333e-05, 9.08832460e-05,
9.24351335e-05, 9.42668510e-05, 9.43254489e-05, 9.50635913e-05,
9.55241224e-05, 9.55978144e-05, 9.70108741e-05, 9.75011193e-05,
9.76820119e-05, 9.76820119e-05, 1.00309605e-04, 1.01024099e-04,
1.01362488e-04, 1.02709762e-04, 1.03470576e-04, 1.03470576e-04,
1.03470576e-04, 1.03470576e-04, 1.03470576e-04, 1.03470576e-04,

1.04577945e-04, 1.04594316e-04, 1.05461842e-04, 1.06017001e-04,
1.06645333e-04, 1.06660336e-04, 1.07150271e-04, 1.07535491e-04,
1.08801011e-04, 1.08916395e-04, 1.09380528e-04, 1.09492673e-04,
1.09741520e-04, 1.10500157e-04, 1.12667960e-04, 1.12756396e-04,
1.12784129e-04, 1.13475838e-04, 1.14240347e-04, 1.14414579e-04,
1.14460972e-04, 1.14517671e-04, 1.14967306e-04, 1.14967306e-04,
1.14967306e-04, 1.15179977e-04, 1.15337742e-04, 1.15734983e-04,
1.15958404e-04, 1.16404397e-04, 1.16404397e-04, 1.16846579e-04,
1.16966738e-04, 1.17212761e-04, 1.17247420e-04, 1.17362458e-04,
1.18662684e-04, 1.19228915e-04, 1.19757611e-04, 1.19890823e-04,
1.20128377e-04, 1.20235597e-04, 1.20595076e-04, 1.20595076e-04,
1.20759850e-04, 1.20831765e-04, 1.21594603e-04, 1.22631793e-04,
1.23179257e-04, 1.23179257e-04, 1.23179257e-04, 1.23282249e-04,
1.23459209e-04, 1.23544360e-04, 1.24964463e-04, 1.25311205e-04,
1.26412015e-04, 1.26454561e-04, 1.27342259e-04, 1.27926856e-04,
1.28311726e-04, 1.28311726e-04, 1.29338219e-04, 1.29338219e-04,
1.29338219e-04, 1.29338219e-04, 1.29788875e-04,
1.30184221e-04, 1.30644666e-04, 1.30740559e-04, 1.30775311e-04,
1.31174665e-04, 1.31391207e-04, 1.31689823e-04, 1.32757307e-04,
1.33076436e-04, 1.33120039e-04, 1.33290221e-04, 1.33757083e-04,
1.33780138e-04, 1.34128524e-04, 1.34128524e-04, 1.34128524e-04,
1.34128524e-04, 1.34162023e-04, 1.35886015e-04, 1.35978572e-04,
1.36111885e-04, 1.36278319e-04, 1.36771450e-04, 1.37601396e-04,
1.37960767e-04, 1.37960767e-04, 1.37960767e-04, 1.37960767e-04,
1.37960767e-04, 1.38439798e-04, 1.39603157e-04, 1.39715010e-04,
1.39876889e-04, 1.40216202e-04, 1.40297066e-04, 1.40834950e-04,
1.41096239e-04, 1.41096239e-04, 1.41096239e-04, 1.41770534e-04,
1.41859026e-04, 1.43521278e-04, 1.43709133e-04, 1.43946347e-04,
1.44035744e-04, 1.44092492e-04, 1.45920042e-04, 1.47117676e-04,
1.47158152e-04, 1.47160380e-04, 1.47275337e-04, 1.47599795e-04,
1.48884864e-04, 1.50298168e-04, 1.50328648e-04, 1.50488905e-04,
1.50502655e-04, 1.50894589e-04, 1.51756844e-04, 1.52162611e-04,
1.52321114e-04, 1.52508435e-04, 1.52553712e-04, 1.52924766e-04,
1.53389729e-04, 1.53824420e-04, 1.54202181e-04, 1.56304816e-04,
1.56818649e-04, 1.57053552e-04, 1.57455224e-04, 1.57616257e-04,
1.57780482e-04, 1.59643088e-04, 1.59644744e-04, 1.60255712e-04,
1.60372339e-04, 1.61364257e-04, 1.61672774e-04, 1.61912290e-04,
1.63305833e-04, 1.64239009e-04, 1.64270438e-04, 1.66063887e-04,
1.67314646e-04, 1.68625919e-04, 1.68921261e-04, 1.68932563e-04,
1.70102542e-04, 1.72045455e-04, 1.72195276e-04, 1.72439875e-04,
1.72505968e-04, 1.72600267e-04, 1.73723533e-04, 1.73920712e-04,
1.74879297e-04, 1.74924370e-04, 1.76914396e-04, 1.77238979e-04,
1.77821907e-04, 1.78505159e-04, 1.79636416e-04, 1.79948827e-04,
1.81926287e-04, 1.83682009e-04, 1.83947690e-04, 1.86819455e-04,
1.88310570e-04, 1.89533129e-04, 1.89738323e-04, 1.90336920e-04,
1.91091512e-04, 1.91389018e-04, 1.91454601e-04, 1.91612177e-04,
1.94007329e-04, 1.94007329e-04, 1.94007329e-04, 1.94007329e-04,
1.94493563e-04, 1.95444420e-04, 1.95692847e-04, 1.96305933e-04,
1.96384845e-04, 1.96490696e-04, 1.96525507e-04, 1.97168930e-04,

```

1.97771140e-04, 1.98566551e-04, 2.00788503e-04, 2.02638135e-04,
2.05895748e-04, 2.07180666e-04, 2.07411472e-04, 2.09031466e-04,
2.09827310e-04, 2.10174607e-04, 2.10174607e-04, 2.10216255e-04,
2.11252425e-04, 2.11617132e-04, 2.12371139e-04, 2.12727335e-04,
2.13027656e-04, 2.13460021e-04, 2.14104343e-04, 2.14385321e-04,
2.15493739e-04, 2.16159728e-04, 2.16798579e-04, 2.17855406e-04,
2.20129827e-04, 2.20344191e-04, 2.21722662e-04, 2.21758956e-04,
2.21836457e-04, 2.22077987e-04, 2.24745144e-04, 2.26118887e-04,
2.26341884e-04, 2.27942294e-04, 2.29424780e-04, 2.29934612e-04,
2.31304659e-04, 2.34115242e-04, 2.35449765e-04, 2.36932984e-04,
2.45263587e-04, 2.49211085e-04, 2.51099048e-04, 2.51790376e-04,
2.52587122e-04, 2.53608377e-04, 2.55536575e-04, 2.57728811e-04,
2.58768560e-04, 2.59475786e-04, 2.61603026e-04, 2.62782414e-04,
2.63307904e-04, 2.63353003e-04, 2.65722523e-04, 2.68416725e-04,
2.70009644e-04, 2.71470911e-04, 2.71635343e-04, 2.71823599e-04,
2.74379317e-04, 2.78105022e-04, 2.78852248e-04, 2.81607962e-04,
2.83093230e-04, 2.85640884e-04, 2.88069181e-04, 2.92104296e-04,
2.92537642e-04, 2.95106704e-04, 2.95260486e-04, 2.99670849e-04,
2.99894713e-04, 3.03060885e-04, 3.03510209e-04, 3.05017693e-04,
3.05371327e-04, 3.08005434e-04, 3.21848858e-04, 3.24528642e-04,
3.25971249e-04, 3.28856288e-04, 3.29914719e-04, 3.36115063e-04,
3.36279371e-04, 3.39162452e-04, 3.39897532e-04, 3.40567501e-04,
3.42622690e-04, 3.43922084e-04, 3.44901919e-04, 3.45585705e-04,
3.53745557e-04, 3.54374938e-04, 3.57440356e-04, 3.68069316e-04,
3.68731506e-04, 3.82226127e-04, 3.86502143e-04, 3.90829083e-04,
3.97848021e-04, 4.00795115e-04, 4.01467027e-04, 4.08351701e-04,
4.14336320e-04, 4.16248036e-04, 4.16956698e-04, 4.25664482e-04,
4.29381773e-04, 4.30065577e-04, 4.41439298e-04, 4.43262185e-04,
4.45679457e-04, 4.56635672e-04, 4.57151816e-04, 4.59046973e-04,
4.64592146e-04, 4.70087732e-04, 4.74567838e-04, 4.78528593e-04,
4.82097805e-04, 4.82379708e-04, 4.87934902e-04, 4.91320955e-04,
5.13716150e-04, 5.20664881e-04, 5.30854359e-04, 5.35320896e-04,
5.57268608e-04, 5.79134374e-04, 5.83578196e-04, 5.85961232e-04,
5.89367112e-04, 6.00972199e-04, 6.18029260e-04, 6.36245190e-04,
6.39175621e-04, 6.39658698e-04, 6.50386475e-04, 6.52666755e-04,
6.53304236e-04, 6.57584454e-04, 6.72125043e-04, 6.75884070e-04,
6.96773565e-04, 7.02779693e-04, 8.02389918e-04, 8.09765155e-04,
8.27680639e-04, 8.28161684e-04, 8.91679944e-04, 9.06905177e-04,
9.11030947e-04, 1.06932727e-03, 1.16014793e-03, 1.22121842e-03,
1.28518223e-03, 1.31649255e-03, 1.33454696e-03, 1.60381340e-03,
1.66271922e-03, 1.83294958e-03, 1.83956000e-03, 1.85901075e-03,
1.91601907e-03, 2.06858567e-03, 2.09156784e-03, 2.15413734e-03,
2.40958731e-03, 2.61061954e-03, 2.95960729e-03, 3.19377357e-03,
4.10535944e-03, 4.51428291e-03, 5.02308256e-03, 1.00733591e-02,
1.04190956e-02, 1.11139914e-02, 2.66036110e-02, 9.56531268e-02,
1.01851708e-01])

```

```

from sklearn.model_selection import GridSearchCV
clf_tree = DecisionTreeClassifier(random_state=2)

```

```

grid = GridSearchCV(
    estimator=clf_tree,
    param_grid={
        'criterion': ['gini', 'entropy'],
        'max_depth': [12, 14, 16, 18, 20, 22, 24, 26, 28, 30],
        "ccp_alpha": [0.001, 0.005, 0.01, 0.02, 0.03, 0.04, 0.05]
    }
)

grid.fit(X_train, Y_train)

GridSearchCV(estimator=DecisionTreeClassifier(random_state=2),
              param_grid={'ccp_alpha': [0.001, 0.005, 0.01, 0.02, 0.03,
                                         0.04, 0.05],
                           'criterion': ['gini', 'entropy'],
                           'max_depth': [12, 14, 16, 18, 20, 22, 24, 26,
                                         28, 30]})

print(grid.best_params_)

{'ccp_alpha': 0.001, 'criterion': 'entropy', 'max_depth': 26}

dt_clf1 = DecisionTreeClassifier(criterion = 'entropy', max_depth =
0.26, ccp_alpha = 0.001)
dt_clf1.fit(X_train, Y_train)

DecisionTreeClassifier(ccp_alpha=0.001, criterion='entropy',
max_depth=26)

dt_clf1.score(X_train, Y_train)

0.8905798663505066

dt_clf1.score(X_test, Y_test)

0.8832557337471978

```

1. Applying Random Forest

```

from sklearn.ensemble import RandomForestClassifier

# n_estimators = Number of Decision Trees
# bootstrap = This is random sampling with replacement
# max_depth = depth of each decision tree
# max_features = total number of features selected in each sample
rf_clf = RandomForestClassifier(bootstrap = True, n_estimators = 50,
max_depth = 15,
                                random_state = 2, max_features = 10)

rf_clf.fit(X_train, Y_train)

```

```
RandomForestClassifier(max_depth=15, max_features=10, n_estimators=50,  
                        random_state=2)
```

```
rf_clf.score(X_test, Y_test)
```

```
0.9387825487152958
```

1. Evaluating Model Performance And Confusion Matrix

1. Logistic Regression

```
from sklearn.metrics import confusion_matrix, accuracy_score,  
classification_report
```

```
y_pred_Logistic = logit_model.predict(X_test)
```

```
print(classification_report(Y_test, y_pred_Logistic))
```

	precision	recall	f1-score	support
0	0.91	0.81	0.86	2920
1	0.83	0.92	0.87	2879
accuracy			0.86	5799
macro avg	0.87	0.87	0.86	5799
weighted avg	0.87	0.86	0.86	5799

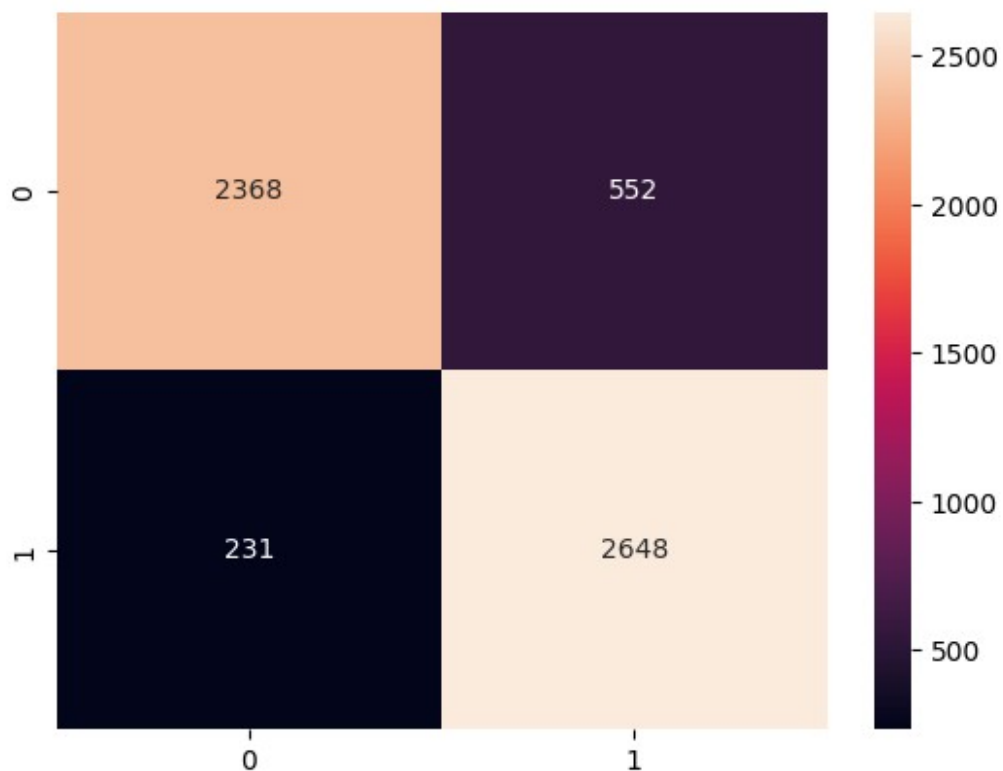
This is matrix between Actual Value and Predicted Value

```
confusion_matrix(Y_test, y_pred_Logistic)
```

```
array([[2368,  552],  
       [ 231, 2648]])
```

```
sns.heatmap(confusion_matrix(Y_test, y_pred_Logistic), annot = True,  
fmt = '0.0f')
```

```
<Axes: >
```

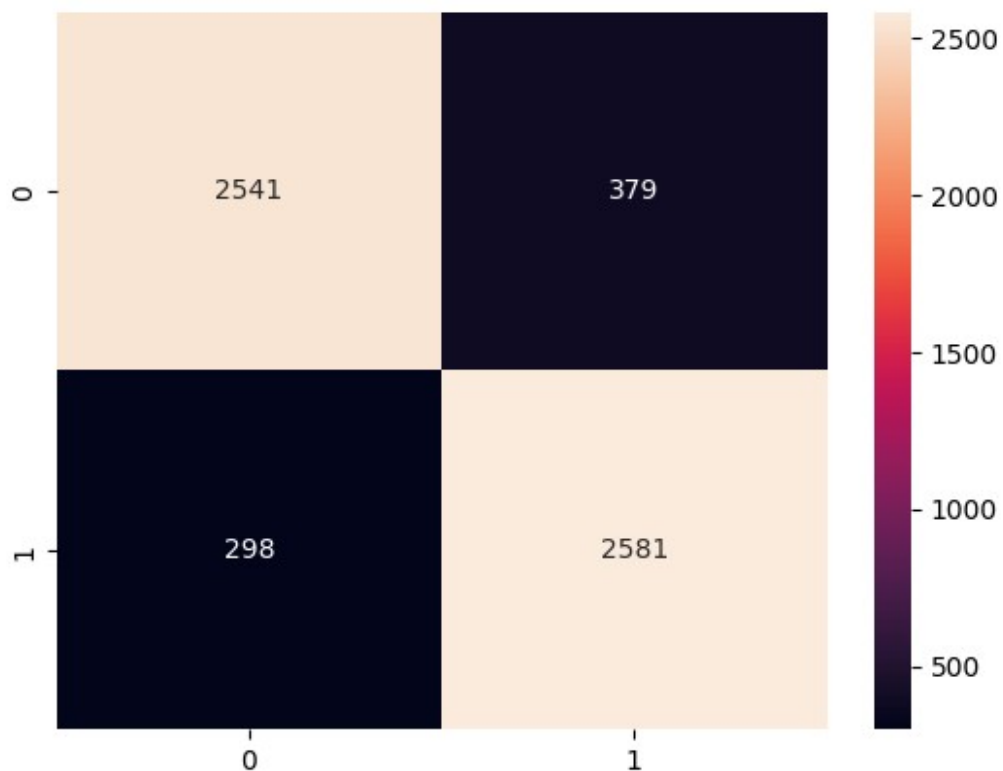


1. Decision Tree

```
ypred_DTree = dt_clf1.predict(X_test)
print(classification_report(Y_test, ypred_DTree))
```

	precision	recall	f1-score	support
0	0.90	0.87	0.88	2920
1	0.87	0.90	0.88	2879
accuracy			0.88	5799
macro avg	0.88	0.88	0.88	5799
weighted avg	0.88	0.88	0.88	5799

```
sns.heatmap(confusion_matrix(Y_test, ypred_DTree), annot = True, fmt =
'0.0f')
<Axes: >
```

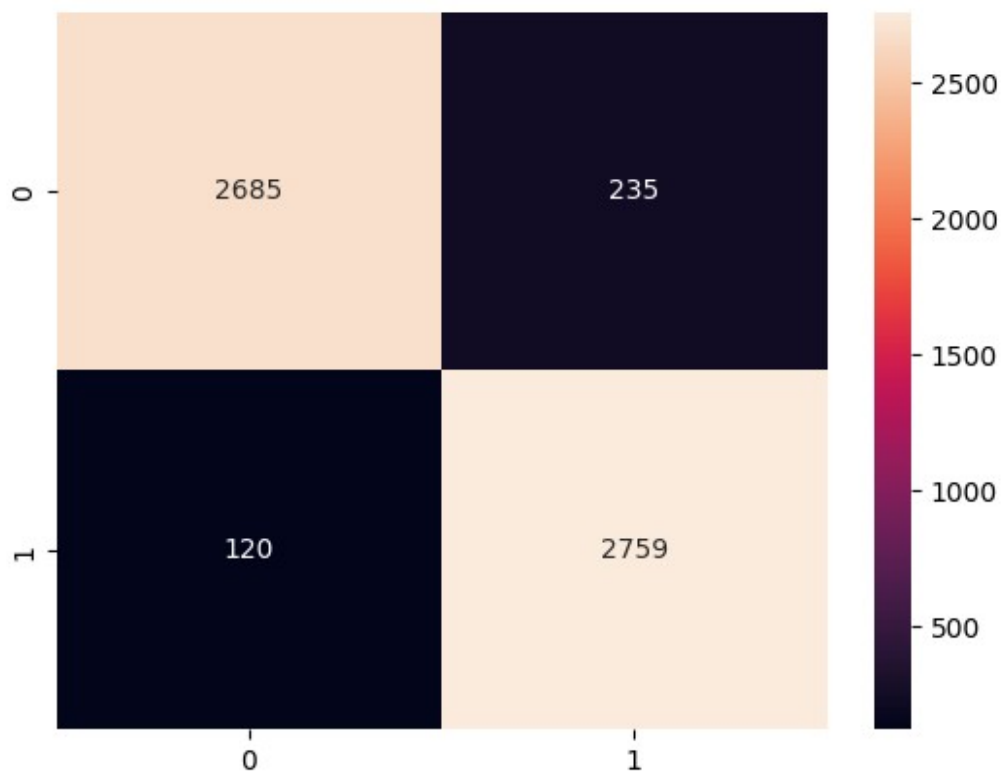


1. Random Forest

```
ypred_RandomForest = rf_clf.predict(X_test)
print(classification_report(Y_test, ypred_RandomForest))
```

	precision	recall	f1-score	support
0	0.96	0.92	0.94	2920
1	0.92	0.96	0.94	2879
accuracy			0.94	5799
macro avg	0.94	0.94	0.94	5799
weighted avg	0.94	0.94	0.94	5799

```
sns.heatmap(confusion_matrix(Y_test, ypred_RandomForest), annot =
True, fmt = '0.0f')
<Axes: >
```



1. Most Important Features Using Random Forest

```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': rf_clf.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

feature_importance.head(20)
```

	Feature	Importance
31	BasePolicy	0.186256
11	Fault	0.172237
3	Make	0.064288
13	VehicleCategory	0.061034
22	AgeOfVehicle	0.059003
12	PolicyType	0.054921
15	PolicyNumber	0.040184
9	MaritalStatus	0.037386
10	Age	0.036387
18	DriverRating	0.032029
21	PastNumberOfClaims	0.027856

5	DayOfWeekClaimed	0.027657
16	RepNumber	0.026011
23	AgeOfPolicyHolder	0.021863
14	VehiclePrice	0.020029
2	DayOfWeek	0.018305
6	MonthClaimed	0.017012
0	Month	0.016705
27	NumberOfSupplements	0.014902
7	WeekOfMonthClaimed	0.014491

```

print("Top 10 Important Features:")

for i, row in feature_importance.head(10).iterrows():
    print(
        f"{row['Feature']}: {row['Importance']:.4f}"
    )

```

Top 10 Important Features:

BasePolicy: 0.1863

Fault: 0.1722

Make: 0.0643

VehicleCategory: 0.0610

AgeOfVehicle: 0.0590

PolicyType: 0.0549

PolicyNumber: 0.0402

MaritalStatus: 0.0374

Age: 0.0364

DriverRating: 0.0320

1. Comparing The Model Performance

```

results = pd.DataFrame({
    'Model': [
        'Logistic Regression',
        'Decision Tree',
        'Random Forest'
    ],
    'Accuracy': [
        0.86,
        0.88,
        0.94
    ],
    'Precision': [
        0.87,
        0.88,
        0.94
    ],
    'Recall': [
        0.87,
        0.88,

```

```

        0.94
    ],
    'F1 Score': [
        0.86,
        0.88,
        0.94
    ]
})

```

results

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.86	0.87	0.87	0.86
1	Decision Tree	0.88	0.88	0.88	0.88
2	Random Forest	0.94	0.94	0.94	0.94

```

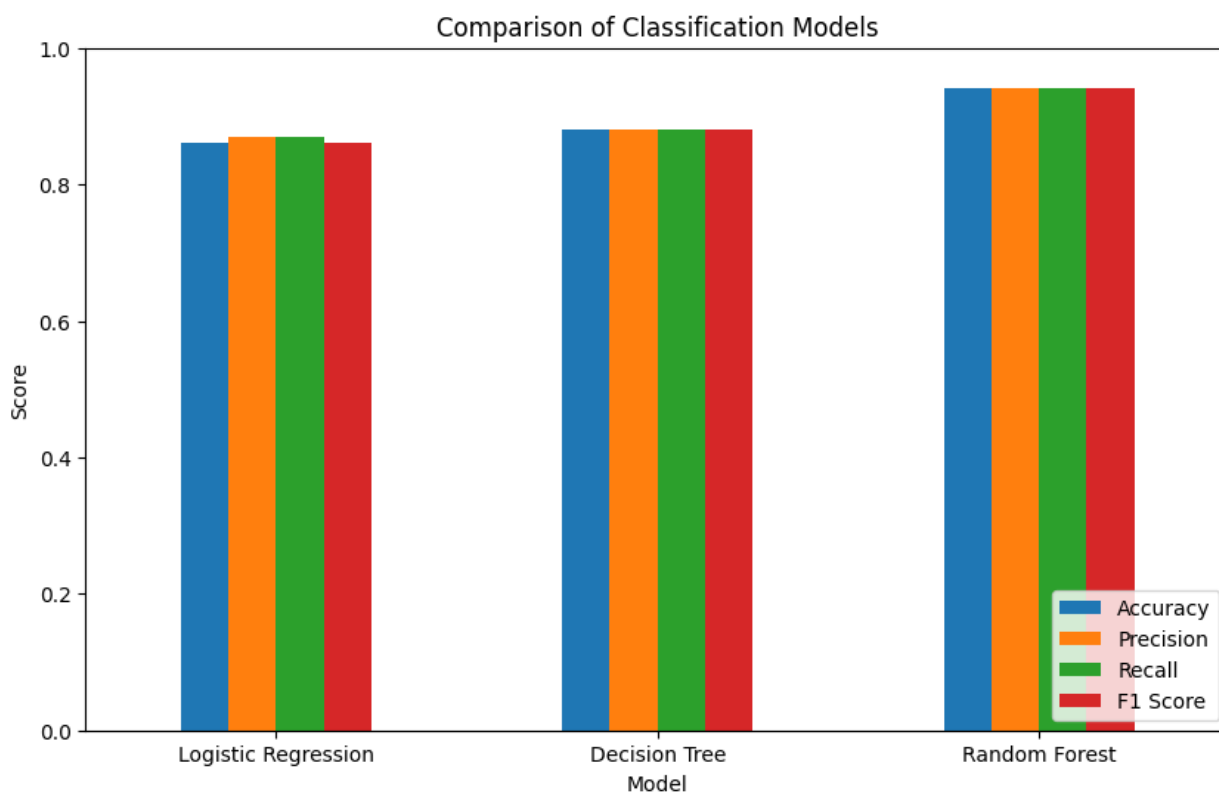
results.set_index('Model').plot(
    kind='bar',
    figsize=(10, 6)
)

plt.title('Comparison of Classification Models')
plt.xlabel('Model')
plt.ylabel('Score')
plt.ylim(0, 1)

plt.xticks(rotation=0)
plt.legend(loc='lower right')

plt.show()

```



Comparison of Classification Models

The three classification models — Logistic Regression, Decision Tree, and Random Forest — were evaluated using Accuracy, Precision, Recall, and F1 Score.

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	86.50%	87%	87%	86%
Decision Tree	88.33%	88%	88%	88%
Random Forest	93.88%	94%	94%	94%

The results show that Random Forest achieved the highest overall performance among the three evaluated models. It achieved an accuracy of approximately 93.88%, compared with 88.33% for Decision Tree and 86.50% for Logistic Regression.

Random Forest also achieved the highest Precision, Recall, and F1 Score. Therefore, based on the evaluation results obtained on the test dataset, Random Forest provides the strongest predictive performance among the three models evaluated in this study.

17. Key Insights

The following key insights were obtained from the analysis:

1. **Random Forest achieved the highest overall performance.** It achieved an accuracy of 93.88%, which was higher than Decision Tree (88.33%) and Logistic Regression (86.50%).
2. **Random Forest provided the highest fraud detection capability.** For the fraud class, Random Forest achieved a recall of 96%, compared with 92% for Logistic Regression and 90% for Decision Tree.
3. **Random Forest achieved higher fraud precision.** The fraud precision of Random Forest was 92%, meaning that most claims predicted as fraudulent were actually fraudulent.
4. **The F1 Score supports the Random Forest result.** Random Forest achieved an F1 Score of 94% for the fraud class, indicating a strong balance between precision and recall.
5. **Decision Tree performed better than Logistic Regression.** Decision Tree achieved 88.33% accuracy compared with 86.50% for Logistic Regression.
6. **The results demonstrate the importance of handling class imbalance.** SMOTE was applied to the training data to balance the target classes before training the classification models.
7. **Fraud detection should focus on both financial protection and operational efficiency.** Identifying fraudulent claims can help reduce potential financial losses, while maintaining good precision can help avoid sending excessive numbers of legitimate claims for manual investigation.

Recommended Model for Fraud Detection

Based on the evaluation results obtained from the test dataset, **Random Forest is the recommended model for this fraud detection assignment.**

Random Forest achieved:

- **93.88% Accuracy**
- **94% Precision**
- **94% Recall**
- **94% F1 Score**

For the fraud class specifically, Random Forest achieved:

- **92% Precision**
- **96% Recall**
- **94% F1 Score**

The high recall for the fraud class is particularly relevant to the insurance business because failing to identify an actual fraudulent claim may result in financial losses. The high precision is also valuable because it can help reduce the number of legitimate claims incorrectly sent for fraud investigation.

Therefore, based on the experimental results, Random Forest provides the most effective combination of fraud detection capability and prediction accuracy among the three models tested.

However, the model should be further validated on new and unseen insurance claim data before being deployed in a real-world insurance environment. In a production system, the decision threshold could also be adjusted according to the company's tolerance for missed fraud cases versus unnecessary investigations.

Final Conclusion

The objective of this project was to develop classification models for detecting fraudulent insurance claims using the Fraud Oracle dataset.

The dataset was cleaned and analyzed through exploratory data analysis. Categorical variables were encoded, the data was divided into training and testing sets, and SMOTE was applied to the training data to address class imbalance. Three classification algorithms — Logistic Regression, Decision Tree, and Random Forest — were then trained and evaluated.

Among the three models, Random Forest produced the strongest results, achieving approximately 93.88% accuracy and 94% for precision, recall, and F1 Score. For the fraud class, it achieved 92% precision and 96% recall.

From a business perspective, these results indicate that the Random Forest model can effectively identify a high proportion of potentially fraudulent claims while maintaining a relatively high level of precision. This can support insurance companies in prioritizing suspicious claims for further investigation, potentially reducing financial losses associated with fraudulent claims and improving the efficiency of the claims-review process.

Therefore, based on the results of this experiment, **Random Forest is recommended as the model for fraud detection among the three models evaluated.** Further validation using new real-world claims data would be required before production deployment.